LiquidAIty State of the App and Launch Plan

Independent review packet for the owner and GPT Web

Prepared 2 November 2026 in America New York time. Author: Codex, integrating two read only audit agents and primary source research. Repository snapshot: 05285ac6227dfd438ba89f6180e669c9e3e1655a, titled Checkpoint Bot Mode working reasonably well.

Executive judgment: the current local Main and directly addressed agent chat baseline works well enough to stop rebuilding that baseline and proceed to bounded feature completion. The application is not yet proven ready for a public multi user launch. The remaining work is specific and testable: truthful profile controls, Project isolation, history selection and retrieval, graph workflow completion, and release acceptance.

This report is a decision and engineering review artifact. Its proposal section is not permission to implement changes. It does not authorize a new runtime, a database replacement, a plugin installation, a deployment, a Git operation, or deletion of user data.

The full original conversation and graph data were preserved during this audit. Previously authorized live tests are identified as tests. No further model tests, production repairs, profile changes, service restarts, migrations, or installations were performed for report preparation. The owner separately authorized a repository design-understanding skill; its instruction files and a reference from AGENTS are the only non-report edits in this stage. They do not alter application execution.

Reading order: read Chapters 1 through 4 for the verdict and evidence; Chapters 5 through 19 for the architecture audit; Chapters 20 through 27 for external comparisons and the ecosystem; Chapters 28 through 37 for proposals, the waterfall plan, launch gates, decisions, and the independent reviewer packet. Appendices identify source anchors, measurements, and methodology. The accompanying CSV inventories all 762 named plugin declarations in the inspected Hermes catalog; it is not a claim to have executed or security audited all 762 implementations.

Contents

Contents 2

1 Executive verdict 4

2 Scope evidence and limits 5

3 Owner decisions that control the remaining work 6

4 Current test results and latency 7

5 The actual application ownership map 9

6 Main submission and reply handling 10

7 Profiles Cards and setting parity 11

8 Conversation storage and the database question 11

9 History selection and continuous conversation 13

10 Retrieval and the graph memory product 14

11 ThinkGraph and KnowGraph completion 15

12 CodeGraph and structural understanding 15

13 Project and user isolation 16

14 Scale and the ten million Project question 17

15 Authentication privacy and public boundaries 17

16 Startup packaging and operational truth 18

17 Domain modules and optional release scope 19

18 Voice images approval and stop semantics 19

19 Runtime conclusion and the boundary to freeze 20

20 Hermes Desktop dashboard and Bot Mode 20

21 Open WebUI comparison 21

22 LibreChat comparison 22

23 AnythingLLM comparison 23

24 Letta comparison 23

25 Dify comparison 24

26 Community Hermes interfaces and the transplant risk 24

27 Hermes plugin catalog and ecosystem positioning 25

28 Code removal rejected work and cost of churn 27

29 Feature and idea preservation audit 29

30 Known work versus owner decisions 30

31 Waterfall stage zero and profile truth 31

32 Waterfall history and interaction stages 32

33 Waterfall graph and memory completion 33

34 Waterfall isolation and release stages 33

35 Waterfall optional expansion and ecosystem release 34

36 Launch proposal register 35

37 Review instructions and the next decision 38

Appendix A Repository evidence registry 40

Appendix B External evidence and research snapshots 45

Appendix C Acceptance matrix for the first release 49

Appendix D Audit change record and limitations 51

1 Executive verdict

Yes, there is now a working ordinary chat execution baseline. The owner observed Main and Builder responding, and the subsequent authorized repeat tests produced Main answers and separately attributed replies from every one of the nine connected profiles. Both application builds succeeded at the audited checkpoint. Eight focused chat and completed pair storage tests passed. The application remained open in Preview. This is a materially stronger state than a UI that renders but cannot deliver a real model turn. [R01 R02 R36 R37]

No, that is not the same as a finished runtime product or a launch ready application. A direct @Name response is not proof of Main independently choosing message_agent, the recipient doing useful work, and a later response returning to the foreground conversation. A stored model choice is not proof the next Hermes turn uses it. A queued graph write is not proof the record exists. A process health response is not proof every tool or provider is ready. Those distinctions drive the conclusions throughout this report.

The recommended decision is to preserve the current checkpoint and stop broad runtime reconstruction. Continue with a short sequence of owner local feature and acceptance stages. The first stage should establish that the Card UI edits the exact Hermes profile used for the next turn. The second should make the selected history policy real at the provider request boundary. The third should finish the approved storage to Think workflow and truthful graph refresh. Project and account isolation must be solved before claiming safe multi Project or multi user operation.

What can proceed now

Single owner, single active Project development can proceed on top of the demonstrated chat path. UI improvements that do not change execution ownership, deliberate graph workflow completion, useful saved profile prompts, retrieval experiments, and narrowly scoped domain work can be built and accepted one at a time. The tested baseline is valuable. Its limitations do not justify discarding it.

The relevant restriction is honesty about scope. A local preview can be useful without being a cloud service. A research agent can be useful without live trading execution. A real building modeler for one configured workspace can be useful without arbitrary customer provisioning. The owner should choose the release scope; neither an old roadmap nor an assistant should make every optional domain a prerequisite for any release.

What must not be claimed yet

The evidence does not establish a six second maximum response time, strict last three pair input, automatic source linked Think creation on every new conversation pair, fully verified live Know ingestion, all profile editor fields taking effect, general asynchronous Main delegation presentation, public account safety, or large scale tenant isolation. These are not all runtime failures. They are distinct unfinished capabilities or acceptance gaps.

The current architecture still has a concrete cross Project coupling: the ordinary application opens the same Bot Chat by profile, while roster and Project specific MCP credentials are written into profile level configuration. Multiple Projects sharing an agent therefore require more than a new conversation label. The existing cross owner profile guard rejects ambiguity; it must not be weakened to conceal this limitation. [R01 R05 R23 R25]

The move on decision

The answer to the owner's central question is **move on from rebuilding basic chat, but not from verifying the product boundaries around it**. Freeze what is demonstrated. Treat each remaining stage as a bounded change with an observable exit criterion. If a proposed change again expands into another session manager, scheduler, reply poller, or graph store, stop and require a precise explanation of the missing Hermes or provider capability before proceeding.

2 Scope evidence and limits

The audit combines current first party source, installed Hermes source, the current saved profile configuration where safely readable, read only SQLite metadata, Git history, existing test definitions, authorized earlier live tests, official documentation, and pinned external repository research. Two support agents independently examined application state and external comparators. The Primary retained responsibility for source reconciliation, all report writing, and final claims.

The application source was checked at commit 05285ac6227dfd438ba89f6180e669c9e3e1655a. At the start of reporting, the only tracked changes were the automatically maintained CBM artifact and the existing per user Engraphis SQLite database, WAL, and shared memory files updated by the authorized conversation tests. No uncommitted application source repair was hidden inside the audit. Subsequent report outputs are separate from production changes.

Evidence classes

Label	Meaning	What it does not prove
Observed live	A real loaded profile received input and produced a recorded response	Every tool, delegation, or domain capability works
Source confirmed	The current implementation and relevant callers were inspected	The deployed process executed it successfully
Test executed	A named deterministic test or build completed at the stated snapshot	Real model judgment or production scale
Test defined	A relevant test exists in source	That it currently passes
Historical	A prior dated report or commit records an outcome	The current path still has the same behavior
Inference	A conclusion follows from inspected relationships	A new incident was reproduced live
Proposal	An option for future approved work	Authorization or completed implementation

Source labels and exact anchors are collected in the evidence registry. External URLs and repository pins are identified in the comparison chapters. The reader should be able to challenge a claim without accepting the author's confidence as evidence.

What exhaustive means here

This is a broad product and architecture review, not a formal proof of every source line, dependency, concurrency interleaving, or third party plugin. The report covers the app's major owners and user promises, all connected profile smoke checks, the complete local plugin declaration inventory, selected high relevance plugin implementation boundaries, and representative external systems. It does not claim a penetration test, a security audit of 762 remote repositories, a load test for 10,000 users, or live acceptance of all optional domains.

The CBM CLI supplied a directional graph map for the canonical C-Projects-LiquidAIty-main project. Its reported generation was 2026-11-02T05:01:22Z, with metadata recorded at 05:01:44Z. Several relevant files reported metadata_changed. Those files were directly read; no reindex, cache access, or lifecycle repair was performed. A zero graph result was never used as proof that a behavior did not exist.

Why older documents are not sufficient

PLAN and ARCHITECTURE contain valuable requirements and historical receipts, but some statements conflict with current source. For example, PLAN describes per user Engraphis database selection as unfinished while current Python selects db/users/<owner>/engraphis.sqlite. PLAN also contains incompatible statements about ordinary per turn graph map delivery. A historical successful test cannot settle either discrepancy. The report states current source and observed behavior separately and recommends targeted documentation reconciliation, not restoration of removed code. [R10 R35]

3 Owner decisions that control the remaining work

The latest owner directions replace earlier exploratory ideas where they conflict. The report treats the following as settled constraints, rather than questions to reopen during implementation.

The Card is the UI for the actual Hermes profile. Main, Builder, ThinkGraph, KnowGraph and the other agents must retain separate profile and execution identities. Saved wires determine the relevant roster. LiquidAIty is a Hermes based UI and Project workbench, not a replacement agent runtime.

Ordinary chat must not repeat application owned conversation discovery, explicit resume, and initial history download before every submission. Hermes may still perform its own required attachment or internal transcript work. Those operations must be named accurately. Lookup, attachment, display history, model replay and creation are different operations.

The latest selected input direction is the last **three completed question answer pairs**, not two, plus the current request and relevant retrieved or graph context. This is a history selection policy, not a reduction of the model's advertised context capacity. The complete conversation should remain durable and retrievable. This policy is not yet implemented in the tested provider input.

The user should not have to stop a chat, discard ideas, or start a new conversation because the model context grew. Existing Hermes steering or admission behavior should support continuing interaction. No fake progress answer should substitute for real execution, and no long pre reply summarization stall should be normalized as the desired user experience.

Main and Bot Mode are not the owners of Think extraction. Application conversation storage is the source. New exact saved pairs should feed the accepted chunking, extraction and settlement owners through the separately approved storage side path. KnowGraph remains ordinary Bot Mode research. A research flag is data, not permission to add an automatic research dispatcher, request registry, personal context extractor, or extra graph tools.

Engraphis owns ThinkGraph and raw Project conversation source. Graphiti owns sourced Knows. CBM owns repository structure. AGE records topology and observations; it is not another execution scheduler. Preserve the tested vendor modifications for Thinks, Knows and relationships without treating the vendor trees as broad cleanup targets.

The user authorized this report, research, agents, and proposals. That authorization is not approval to implement the proposals, install plugins, update Hermes, change the data layout, delete history, create a backup, commit, push, or deploy. Every implementation stage below has an explicit approval boundary.

4 Current test results and latency

The last authorized Main test after compaction used the exact question OKAY, WHAT DO YOU THINK ABOUT 3 MORE SPACE STOCKS? It generated a substantive comparison, not a canned health response. The current Project was already open. No fresh startup sequence was included in this timing.

Main measurement	Earlier large history run	After rebuild before compaction	Warm run after compaction
Hermes receipt to response dispatch	23.29 seconds	12.63 seconds	12.94 seconds
Provider model call	9.2 seconds	10.8 seconds	12.2 seconds
Input tokens	141285	141783	20401
Output tokens	476	526	525
Reported cache reuse	Not recorded for this comparison	Not recorded for this comparison	20224 of 20401 about 99 percent

The warm run was submitted at 00:25:14.436 local time on 2 November. Hermes logged receipt at 00:25:14.727, request setup at 00:25:15.132, first parsed provider event at 00:25:15.986, first substantive provider progress at 00:25:22.976, provider completion at 00:25:27.284 and response dispatch at 00:25:27.668. First substantive provider progress is not a measured browser paint timestamp. The report does not turn it into one. [R36]

The application reached Hermes in about 0.29 seconds. Hermes reached request setup about 0.41 seconds later. The model call consumed 12.2 seconds. Final model completion to response dispatch took about 0.38 seconds. The model call therefore dominated this run. The data does not support blaming a post response Engraphis save for those twelve seconds.

The compaction turn

The preceding request triggered the saved 32000 token threshold. The first compaction pass took about 51.1 seconds and reduced 466 message records to 47. The remaining estimated request still exceeded the threshold, so Hermes performed a second pass taking about 56.0 seconds and reduced the list to 43 records. The subsequent answer request used 19946 input tokens and took 8.3 seconds at the provider. Total Hermes receive to dispatch time was 116.24 seconds.

This proves that the setting was adopted and compaction materially reduced model input. It also proves that this particular pre reply compaction event was far outside the owner's latency requirement. Both facts matter. It would be false to call this a successful six second speed fix.

Connected agent checks

One new prompt of the form @Name Are you working? Reply in one short sentence. was sent to each connected profile. The declared roster was read from Main's actual profile and checked against the rendered wires. WorldSignals had no saved connection and was not tested. Each listed profile logged its own input and response.

Profile UI name	Receipt to dispatch seconds	Provider call seconds	Input tokens
Builder	5.58	1.8	11839
ThinkGraph	14.67	3.0	316014
KnowGraph	2.79	1.8	56308
Magnetic	2.40	1.9	4331
Team	3.05	1.3	7420
Trader	4.42	1.9	11147
WorldView	3.29	1.7	4196
Analyst	3.59	1.7	10778
Quant	3.30	1.4	4162

These short answers are not controlled comparisons of model quality or speed: models, instructions, context, cache state and output lengths differed. The test establishes live direct addressing and reply presentation. Trader's answer that it was idle is not a transport failure. Analyst's mention of a prior verdict is not evidence that a new analysis was performed. An answer claiming Bot Mode compliance is not itself proof of tool use.

ThinkGraph had a separate context issue: its request included 316014 input tokens. A gateway hygiene compaction continued after its turn hold was released, and later committed a much smaller transcript. This is evidence that oversized history is not confined to Main. It is not permission to clear ThinkGraph's database or override its model.

Builds and deterministic tests

The backend build succeeded through the existing full launcher. The client production build completed in approximately two minutes, retaining warnings about large chunks and browser externalization in the WorldView dependency. Two focused test files passed, eight tests total, covering the current Main route and completed pair storage behavior. Backend health, Python rails health, MCP readiness and the frontend returned HTTP 200 during the authorized rebuild pass.

These are checkpoint specific proofs. No new full regression suite, provider tool exercise, stopped turn, voice recording, graph write or cross Project concurrency test was performed during report preparation. Prior full suite totals in PLAN remain historical evidence.

5 The actual application ownership map

LiquidAIty contains several types of state and execution that should not be collapsed into one word such as runtime. The useful separation is already visible in the code and should be preserved.

Owner	Current responsibility	Boundary to preserve
React and TypeScript UI	Chat, Cards, wires, graphs, inspector, terminal presentation and domain views	No semantic agent planner or graph writer in UI
Node backend	Authenticated transport, Project access, profile bindings, terminal connection and product route composition	No independent model loop or Bot scheduler
Python rails	Saved Card domain operations, graph adapters, deterministic computation and retained Run preparation	Existing owner for Python domain execution
Hermes	Profile execution, model requests, tools, Bot delivery, transcript lifecycle and internal turn control	Do not reimplement these in application wrappers
Engraphis	Project conversation source, Thinks, entities, relations and retrieval	One reasoning memory owner
Graphiti and Neo4j	Sourced Know records and evidence graph	One sourced knowledge owner
CBM	Source structure and code graph	Structural evidence not runtime controller
PostgreSQL and AGE	Accounts, Projects, saved Card configuration, topology and observations	Not another agent execution engine

The source route inventory includes authentication, Projects, decks, profile options, IDD, Main chat, terminals, ThinkGraph, KnowGraph read/write, CodeGraph, Jev focus, Trading, WorldView, WorldSignals and NRGSIM. These routes are feature boundaries, not proof that each feature has completed acceptance. [R18]

There is a retained Run and canonical IDF materialization path for saved Card invocations. The ordinary Bot Chat path submits directly through the Hermes Desktop Gateway client. The latter's successful chat does not prove that Run artifacts, IDF construction, domain execution or automatic graph capture happened. Future work must not create duplicate inputs to force those two paths to look identical.

What is worth keeping

The current app has real reusable product structure: owner checked Projects, saved Card identities, wire topology, separate graph authorities, typed transport, direct profile tools, a visible graph inspector, and established domain surfaces. The report does not recommend a blank repository or a new framework. The engineering task is to make the retained surfaces truthful and complete, while deleting only an explicitly superseded path when a replacement is actually proven.

Why the application is not merely another chat frontend

Its distinctive potential is a Project workbench where a person sees and edits the same agents they use, controls their reachability through wires, and can inspect durable reasoning and sourced evidence connected to exact conversation material. This remains a product proposal where the complete workflow is disconnected. The differentiator is not the number of graphs or controls; it is whether the user can recover an idea, understand its evidence, and continue work without reexplaining everything.

6 Main submission and reply handling

The current path is MainProjectChat to useAgentBuilderMainChat to mainSessionClient to the authenticated Main route to HermesGateway and the receiving profile. The frontend opens the Project once and retains the handles returned by Hermes. It submits unchanged user text and attached images. The backend checks Project access and the receiving profile, then forwards prompt.submit through the existing client. [R01 R02 R03 R04]

The backend route no longer explicitly performs session.list, session.resume and session.history before every chat submission. One session.list helper remains for the preserved Card terminal identity check. Treating that terminal check as proof of a per chat history wait would be inaccurate. Hermes's client can still attach an unattached conversation on its own socket when a request requires it. That is Hermes's implemented protocol behavior, not a new application session manager. [R22]

For an @Name request, the application resolves an exact saved address among the current wired roster. This is structural addressing, not a semantic agent selection policy. Ordinary Main delegation through message_agent is a separate path in Hermes. Neither should be substituted silently for the other.

The response listener filters by receiving profile, conversation and admitted request. It forwards text deltas and the completion. After sending the completion and ending the HTTP response, it calls the existing conversation store asynchronously with the user message and assistant reply as one completed pair. The storage code does not invoke ThinkGraph. The earlier user requirement that storage should not delay model submission is represented in this ordering. [R01 R09]

An unresolved later reply boundary

The listener belongs to the initial HTTP request and is detached when that response closes. Events must match its admission identity. That is good protection against attributing another profile's answer to the current message, but it does not establish how a later independent Main wakeup from an asynchronous Bot reply becomes visible in an idle foreground chat.

The correct next step is a bounded live acceptance scenario: Main actually calls one connected profile, the recipient responds after Main's initial response, and the later result appears under the correct identities. If that fails, the repair belongs at the existing Hermes client and view subscription boundary. A new reply poller, fake completion buffer or separate Bot runtime is not justified by this report.

Submission while work is active

The frontend tracks multiple active streams and does not simply reject every new message because one request is active. It retains per request identity and delegates admission and steering semantics to Hermes. This is source evidence, not a completed concurrency test. The owner's never stop a chat requirement should be tested with real successive user input, not inferred from an enabled Send button.

7 Profiles Cards and setting parity

The owner's model is clear: **the Card is the UI for the real Hermes profile**. There should not be a second profile concept that looks editable but has different effective values. The current source has both saved application configuration and live Hermes profile APIs, and those paths must be reconciled field by field.

Some controls have a direct Hermes path. Card toolset, skill and learning operations forward through cardProfile.routes to the profile API. This is a positive existing seam. Preserve it and verify it rather than replacing it with a broad configuration framework. [R06]

The inspected general Card save path persists prompt, model key, reasoning and related fields through the deck API into Python saved Card revisions and grants. The save returns database readback. The reviewed ordinary Bot open path publishes the Project MCP key and roster, and chat sends text and transport fields. No general model, prompt or reasoning publication was observed on that ordinary save/open/chat path. [R07 R08 R01]

This is not a claim that every control is broken. It is a specific absence of observed publication on the reviewed path and a lack of live parity proof. The report authors traced the frontend handler, transport, Python transaction, revision insertion, deck loading and normal Bot submission. A separate Run preparation path does project saved configuration, but that does not prove ordinary Bot Chat consumes it. Installed profile plugin settings were also checked: no enabled custom project graph plugin supplied the missing behavior.

Required acceptance

Use an existing reversible field and a deliberately chosen profile. Save through the Card UI. Read the effective Hermes profile through its own API. Run the next ordinary message and inspect the effective setting. Verify another profile is unchanged. Repeat for model, prompt, reasoning, tool selection and any context controls promised by the release.

If values disagree, repair the smallest existing save/configuration seam. Do not publish every setting before every message, invent a revision polling loop, or create a second configuration owner. A saved value alone is not acceptance, and a successful model reply alone does not prove it used the saved value.

Decision that remains

The owner has selected the identity model, but implementation still needs a precise field authority map. Application topology and Project permissions can remain product state, while model/provider/profile settings should be truthfully represented through Hermes. The decision is how to reconcile the existing saved revision representation without breaking previously saved Cards. It is not whether to add another profile abstraction.

8 Conversation storage and the database question

Hermes stores each inspected profile's state in its own ordinary SQLite state.db. The Main file was about 5.86 MiB at inspection. This file format is not a model context window, an embedding, or a holographic memory system. The database can retain more text than any one model request needs. Its size alone says little about the time required for inference.

The inspected Main database contained messages and full text indexes, sessions, system prompts, model usage, queued admissions, turn leases, runtime epochs, routing, compaction locks, hygiene state, delegation and worker records, and room related tables. Some tables exist because Hermes supports features that this app does not use. Their presence does not establish that those features were exercised. Removing tables because their names look unnecessary would break a storage contract without solving the model-input policy.

[R26 R27 R40]

Engraphis also uses SQLite, but its database contains workspaces, events, memories, entities, edges, embedding state, vectors, graph indexes and source import/provenance records. The current Python integration resolves the authenticated Project's owner in PostgreSQL and uses db/users/<owner>/engraphis.sqlite. The Project remains a workspace inside that owner's database. This is already implemented in current source, despite older PLAN wording to the contrary. [R10]

Why replacing state.db with Engraphis is not a simple speed fix

The two databases share a storage technology, not a schema or execution API. Hermes expects specific admission, conversation, lock and replay operations. Engraphis expects its own memory and graph operations. Pointing one at the other's file would not implement the missing contracts.

The current system does retain conversation material in two places for different purposes: Hermes's execution transcript and the application's Project conversation source. This is real duplication and deserves a deliberate long-term storage policy. It is not a reason to silently delete either store. A future consolidation would need a supported Hermes transcript backend or an explicitly reviewed change, plus proof that runtime bookkeeping, original-source recall, and other surfaces still work.

The immediate fix for a book-sized prompt is to select what enters the request. The ordinary Hermes transcript loader reads active messages for a conversation; compaction-archived messages are excluded. That is already distinct from a full display-history read, but it is not a strict three-pair selection. [R26 R27]

Application persistence is outside the latest reply critical path

The current Main route ends the response before starting the completed-pair save. The application store transports the pair to Python, which writes the existing Engraphis event ledger. This path does not invoke ThinkGraph extraction. In the latest measured Main run, the provider call consumed 12.2 seconds while application admission and final delivery were subsecond. Those measurements do not support assigning the provider delay to the later application save.

There is nevertheless a scale concern in the storage implementation. The conversation operation currently loads and sorts all matching message/status events even when appending, including calculating the next sequence. Initial UI history likewise returns the entire application conversation. This can grow expensive as a chat becomes very long. It should be measured and then bounded through the existing ledger/index/pagination owner, not solved by adding another conversation store. [R09]

File concurrency and deployment

SQLite allows many readers but one writer at a time per database file. Short transactions and owner-partitioned files can work well for a local or modest service. Multiple servers opening the same file over a network share are a different and unsafe deployment assumption. A client/server database or explicit state-host partitioning is preferable when the workload requires many independent concurrent writers to one shared logical database. That is a deployment decision, not evidence that SQLite caused today's latency. [E14]

Engraphis currently has a module-global service lock and caches each opened owner service until shutdown. That could serialize unrelated owners in one Python rails process. The report recommends a contention measurement before changing it. The proper unit, if correction is needed, is the owning service/connection; no new graph or indexer is implied.

9 History selection and continuous conversation

The selected goal is continuous conversation without an ever-growing replay of all prior text. Keep three recent completed exchanges for local continuity, and make older source recoverable through authorized retrieval. The current question, profile instructions, tools, selected graph context and current operation's necessary tool results remain distinct inputs.

A pair must mean a complete user-led exchange, not two arbitrary database rows. Tool calls, tool results, assistant iterations, steering messages and final answers can make one exchange span several rows. A naive last-six-rows slice can create invalid tool sequences or remove the user question that gives a result meaning. The policy therefore needs a semantic unit consistent with Hermes's actual transcript structure, while the implementation remains structural rather than a keyword planner.

Keeping all user messages verbatim in every request is not indefinitely bounded. It also loses meaning if the answers they refer to are omitted. A message such as use the second option depends on an earlier answer. Preserve all original user messages in durable storage; retain the most recent three complete exchanges inline; retrieve the earlier relevant question and answer together when needed.

What Hermes already provides

The default compressor performs threshold-based compaction and retains a recent section. The saved 32000 threshold is an absolute trigger cap, not a guarantee that every request is below 32000 tokens. The selected lean strategy has its own tail budget and protection rules. min_tail_user_messages is a minimum anchor, not a maximum history count. Changing it to three would not implement the requested last-three-only policy. [R29]

Hermes also exposes ContextEngine.select_context. It can replace the message list for a particular provider request without mutating the durable transcript. This is the kind of existing seam relevant to bounded recent history plus retrieval. It is not a ready-made configured three-pair feature in this checkout. The default implementation does nothing, and invalid or failed selection falls back to the unchanged request. A strict policy would have to address that behavior explicitly rather than assume the existence of a hook is acceptance. [R28]

pre_llm_call is different: the inspected documentation describes it as injection, not replacement of the history list. It can append a graph view, but it does not by itself remove already assembled history. The report specifically rejects using the two interfaces interchangeably.

The never stop a chat requirement

The product requirement is not that arbitrary computation takes zero time. It is that growing history must not force the user to discard work, start a new conversation, or lose the ability to submit or steer. For ordinary conversation, input selection should not require an extra summarization model call before every answer. For long tool-intensive work, the active task's state must remain valid and Hermes's own controls must continue to own admission and interruption.

A complete acceptance test should distinguish: immediate acknowledgement that the UI accepted the user's input; actual Hermes admission; first useful model text; completion; and durable storage. A fake answer saying work has started is not an acknowledgement protocol and must not be used to hide a stalled model.

10 Retrieval and the graph memory product

The reasonable starting proposal is three recent complete pairs plus up to five relevant older source passages, with graph context selected by the existing graph owners. Twenty retrieval candidates can be useful for finding those five passages; twenty candidates need not become twenty passages in the model input. These numbers control different stages and should never be hidden behind one ambiguous memory setting.

An embedding is a numerical representation used to search for semantically related text. The answer model receives selected text and references, not the vector itself. Semantic retrieval can handle paraphrases; lexical or exact identifier matching is useful for tickers, filenames, error strings and quoted requirements. A hybrid strategy can improve recall, but adds work and must be evaluated on the app's actual questions. This is a research lesson, not a proposal to install Qdrant or another store. [E10]

Current source already contains search_source_exchanges, returning five matches by default with a permitted range of one through twenty. It asks Engraphis for up to one hundred episodic candidates and an eight-thousand-token internal budget, then filters for indexed conversation-source chunks. The result includes original source pointers. A read-only count found sixty-seven conversation-source records in the tested Project. This proves a stored index exists, not that every raw conversation has been indexed. [R09 R11]

The ordinary Main route does not call this retrieval function before prompt submission. Its existence must not be described as a complete automatic RAG path. The current source index is normally produced by the accepted extraction/settlement owner, while automatic stored-pair delivery is disconnected. Connecting retrieval responsibly therefore requires proving both source coverage and the receiving model input, not merely adding a search call.

Graph memory and raw source serve different purposes

Thinks should capture useful reasoning and operational knowledge with exact source links. Knows should retain a sourced observation and its evidence, with interpretation separate. Raw conversation remains the faithful source when the user asks what they actually said or when an earlier extraction omitted something. None is an adequate substitute for all the others.

The graph map is a compact view of the visible Entity/Relationship structure at a turn. It is orientation, not evidence that every graph record was read. The approved displayed top Jev result must not silently become a hidden set of extra candidate records. A retrieval failure must not relabel the most recent record as the most relevant one.

Retrieval quality is a release criterion

Evaluate the selected design against original corrections, exact identifiers, old ideas, cross-Project contradictions, dated Knows and source-linked Thinks. Test whether the answer can cite the actual passage and whether no result is reported honestly. A greeting should not require a retrieval model or inject unrelated memories just to fill a quota.

RAG is not a substitute for exhaustive reading when the user asks to audit an entire document or codebase. Long-context research has found cases where complete-source input performs better than retrieval, and retrieval research has found benefits from more final evidence chunks on some workloads. The app should select the appropriate method for the task instead of asserting a universal five-chunk optimum. [E11 E18 E19]

11 ThinkGraph and KnowGraph completion

The approved Think workflow has a meaningful existing foundation: full conversation events, explicit pair reading, chunk snapshots, source indexing, preparation and settlement. These are useful owners worth preserving. The ordinary chat save-triggered handoff was removed and is not connected in the current stage. Successful direct ThinkGraph chat therefore does not prove automatic Think production from a newly stored Main pair. [R09 R11 R14 R35]

The next approved implementation should remain one-way and storage-owned: a newly completed pair becomes available to the existing preparation and extraction path, which persists source-linked Thinks through Engraphis. Main's Hermes completion receipt, usage events, and arbitrary latest-two-row guesses must not become a new pair authority. Do not replay old test history to manufacture apparent automatic success.

KnowGraph is an ordinary saved Hermes profile for research. The retained requirement is one sourced observation per Know, with its source document and exact evidence passage, separate bounded interpretation, and the relevant Entity reference. Queue acceptance is not persistence. The acceptance path must read the actual Know back after processing and show the correct citation in the inspector. No automatic research dispatcher is authorized by a researchRequested flag.

Live graph presentation

The source review found a Project-scoped ThinkGraph event stream, but the only production notification caller found was the route's retirement action. Writes through tools and settlement were not observed to emit that existing notification. KnowGraph's hook refreshes on initial load and confirmed deletion; its claimed change-notification behavior was not found as a corresponding subscription in the inspected hook. [R12 R13]

This is a product gap: a successful committed write may not automatically appear in the open graph. The smallest proposal is to complete the existing provider-commit notification boundary. It is not to add a timer, poll for every queued write, or invent a second event bus. The test must prove the right Project refreshes after real persistence, and an unrelated Project does not.

Preserve the source citation idea

Source chunks and Think-to-source citations survived the recent cleanup. They are not equivalent to the rejected Main-linked handoff wrappers. Removing those wrappers should not silently delete the useful ability to recover the exact original exchange. The report's preservation ledger keeps those capabilities separate so that cleanup does not erase the product's central memory promise.

12 CodeGraph and structural understanding

CBM is the source-structure authority. It can map callers, files, imports and definitions, while source annotations explain stable feature membership. It does not prove runtime success, interpret the user's product intent, or authorize an architecture change. Its watcher metadata is not a reason to pause chat, reinstall an indexer, or repair a provider unrelated to the current task.

The application-published CodeGraph read corridor is different from repository-development CLI access. Saved product profiles should retain only their actual granted operations. Codex development uses the installed CLI under the repository's current rule. Removing one transport from developer practice must not silently remove the product agent's granted provider access.

The report used CBM as a directional accelerator and direct source as the final authority. Runtime and vendor boundaries were inspected directly. A structural query that returned no matching symbol was not taken as

proof of absence; the exact routes, registrations and relevant source were read.

Why more documentation has not automatically solved understanding

Documentation helps only if it is discovered, current, correctly interpreted and connected to proof. This repository contains both strong explicit design instructions and stale current-state descriptions. Merely adding more prose can increase ambiguity if contradictory owners remain described as current.

The separately requested repository skill is designed to change decision behavior: recover the requested observable result, locate its owner, identify the preservation set, classify evidence, and keep alternatives as proposals. It does not replace source inspection or make the agent infallible. It explicitly forbids treating familiar patterns as permission to normalize the user's different design.

The skill is stored at .agents/skills/understand-project-design/SKILL.md with a direct AGENTS reference. This keeps the instruction near the repository and makes it discoverable. It does not add hooks, scripts, runtime interception or another status ledger. The format validator passed. Its behavioral value still requires forward evaluation on realistic requests.

13 Project and user isolation

A reusable agent definition is not a live conversation. The same user can reuse one actual profile across many Projects without needing a cloned profile for every Project. Each Project nevertheless needs distinct conversation state and the correct Project-specific credentials, memory scope, files and wired authority. Shared configuration must not become shared private content.

The current application resolves Bot Chat by the receiving profile and fixed title. Hermes's recipient lookup also finds that profile's Bot Chat. LiquidAIty's Project ID is not part of that recipient lookup. Separate browser Projects therefore do not establish separate Hermes conversation histories. [R01 R23 R25]

There is a second coupling: opening a Project writes its roster and internal MCP credential into the profile's configuration. Another Project using the same profile may have different wires or Project scope. The source consequently exposes a potential last-publication-wins conflict. This audit did not reproduce a cross-Project leak or change two Projects concurrently; it identifies the coupling that must be resolved and tested before such use is supported.

Cross-user profile identity is also currently constrained globally. assertCardProfileOwner requires exactly one owner among references to a profile, and the saved profile/Card binding lacks an account namespace. Default Projects seed fixed profile names. A second owner can therefore encounter a conflict rather than an independent private instance. The conflict rejection is useful protection; disabling it would not implement tenancy. [R05 R08]

Required owner decision

Choose the deployment and profile ownership unit before implementation. A credible design may use user-owned Hermes profile homes with separate Project conversations, while shared templates contain only reusable configuration. Tool credentials and context must remain request/conversation scoped under the supported runtime authority. If Hermes cannot represent a required isolation boundary through existing public interfaces, state that gap plainly and seek approval for the smallest extension. Do not hide it behind a second application session manager.

Acceptance before multiple Projects are claimed

Use two Projects with deliberately different private content, graph sources and wires. Exercise the same human-facing agent name concurrently. Verify the actual Hermes conversation, tool principal, roster and retrieved source scope for each turn. Then repeat with a second user. A title or a different UI tab is not enough. This is a core privacy requirement, not an optional edge case for a hosted product.

14 Scale and the ten million Project question

Ten million saved Projects do not require ten million resident agents. The durable configuration, conversation and memory can be stored while only active Projects consume execution capacity. The owner's requirement that all wired agents are prepared when an open Project becomes usable can coexist with this: prepare that active Project in parallel, not every dormant Project in the system.

The scale model must distinguish registered users, users currently interacting, open Projects, simultaneous model generations, background tasks, database writers and tool processes. Ten thousand simultaneous users are not necessarily ten thousand simultaneous model calls. Conversely, if they all submit at once, the infrastructure and provider must admit or queue that demand honestly; a fast single-user test proves none of this.

For illustration only, if ten thousand active users each send one message per minute, the average arrival rate is about 167 requests per second. At a twelve-second average service time, roughly two thousand requests would be in service on average before adding tool or background work. This is arithmetic under stated assumptions, not a capacity measurement or deployment sizing recommendation.

The current local stack has owner-specific Engraphis files, but a global lock can still serialize operations inside one rails process. Hermes profiles and Bot Chats are not yet Project-isolated. Application authentication lacks demonstrated production abuse controls. None of these facts requires replacing the current local runtime to continue feature work, but all matter before a shared public launch.

A staged scale path

First certify a single-owner local release. Then prove two concurrently active Projects. Then prove two users with negative isolation tests. Next run a bounded concurrent workload using the chosen deployment topology and provider limits. Only after measuring resource use, queueing, errors and recovery should the supported concurrency be advertised. Large-user-number aspirations should inform clean boundaries without becoming an excuse to build a speculative distributed framework today.

15 Authentication privacy and public boundaries

The current app has meaningful authentication protections: cryptographically random session tokens stored as hashes, server-side expiry, HttpOnly cookies, production Secure cookies, SameSite behavior, password hashing, and owner checks. Terminal upgrades check credentials, Origin, Project ownership and the precise profile/conversation relationship. These are existing positive controls, not unnecessary wrappers to remove during performance work. [R16 R17]

The backend binds to loopback. Internal Python rails and graph services rely on the local network boundary rather than browser-style per-request authentication. That is a deliberate local architecture, not a claim that those endpoints are already exposed to the internet. A deployment that publishes them directly would change the security model and needs a separate design and proof.

Before public signup, account abuse and recovery require attention. The reviewed auth routes contain no demonstrated login/signup throttling, general user budget enforcement, verified-email flow or password-recovery endpoint. There may be alternative identity-provider integrations outside the reviewed path, but their presence must be established rather than assumed. Use one chosen identity owner, not an additional shadow user store.

Tool and retrieval authorization

The model's request text is untrusted as an authorization source. User and Project scope must come from the authenticated application. An agent title or a supplied Project UUID does not authorize access. Credentials should be obtained from the correct profile and active scope; process-global configuration must not accidentally select another user's token or workspace during concurrent work.

Vector indexes are not inherently private. Embeddings and retrieved text retain the sensitivity of their source. Retrieval must enforce owner and Project filters before supplying content, and source references must be checked again when expanded. A shared public example Project must not include the author's private history or credentials.

Data deletion and export

The current Project deletion route removes application relational rows, but the reviewed path does not remove all related Engraphis, Graphiti, Hermes or filesystem data. Some data may intentionally be shared or retained. Therefore the UI must not promise total data erasure until exact ownership and retention semantics are defined and implemented. This finding does not authorize deleting anything during this audit.

The release decision must specify archive versus delete, export format, recovery window, shared-resource treatment and backup responsibilities. Test those operations on a deliberately disposable fixture and prove unrelated Projects survive. Do not use the user's actual working Project as the deletion test.

16 Startup packaging and operational truth

The current full launcher is a development workstation path. It uses PowerShell, local Python environments, Docker Compose for the root data services, a backend build, Python rails and MCP readiness, the Vite frontend and companion services. Its readiness ordering is useful: the UI is not started before the required backend/rails/catalog checks. [R19]

The current root scripts do not invoke the optional WorldSignals startup branch in start-dependent-services.ps1. WorldSignals may already be running, but the canonical launch sequence alone does not prove it started that service or supplied its operator credential. The release bundle must explicitly decide whether this module is included or optional.

Process health endpoints are narrower than functional readiness. Backend and rails status responses indicate liveness. MCP exposes more detailed family availability, but a successful readiness response should not be interpreted as every provider and tool being usable by every selected profile. The release test should read actual provider/catalog detail and exercise the promised capabilities.

Local installed product versus hosted service

A production client build is not an installer, a reproducible fresh-machine setup or a supported hosted deployment. A local release needs a documented supported environment, controlled installation and update behavior, credential onboarding, durable data placement, startup failure messages and a recovery rehearsal. A hosted release adds private-service networking, tenant isolation, quotas, shared resource coordination, observability, backups and operator responsibility.

Do not make cloud-scale infrastructure a prerequisite for an honestly scoped local preview. Do not call a local preview a cloud-ready service. This is the first business and release-scope decision in the waterfall plan.

17 Domain modules and optional release scope

Trading

Trading is intentionally constrained in current source. Its contract requires paperOnly true and executionApproved false. It exposes research, decisions, assignments and observations, but does not establish live order execution. The Trader greeting test did not access a broker, start a strategy or place a trade. A research/paper-observation release is a valid scope; autonomous execution requires a separately authorized acceptance project. [R21]

WorldView and WorldSignals

WorldView has an actual mounted visual surface, Project capability controls and an agent bridge. A responding WorldView profile proves chat, not that every map layer, data feed, selected-state read or action works. Test the specific layers and operations promised by the release rather than replacing the renderer when a provider is unavailable.

WorldSignals was disconnected in the tested Project and was deliberately not addressed. Its startup path and feed readiness remain separate from the working Main/Bot baseline. Optional feed failures should be accurately presented without taking down unrelated chat functionality.

NRGSIM

The retained NRGSIM module is a real bounded local integration with saved OpenStudio model files and historical engine outputs. The current routes enforce an operation whitelist, one configured workspace Project, existing OpenStudio prerequisites and per-Project serialization. They are not generalized provisioning for any customer Project or a complete engineering-design product.

Preserve the accepted geometry/modeler and completed simulation evidence. Generalized Project creation, additional system types and arbitrary deployment support should be separate workstreams with domain acceptance, not bundled into a chat-runtime repair. A model reply or a completed process is not proof that engineering assumptions and outputs are correct. [R20]

18 Voice images approval and stop semantics

The Main route retains image attachment forwarding and voice transcription/speech forwarding through the receiving Hermes profile. The broader conversation contains historical image and voice work, but this reporting pass did not record a new microphone input, audible reply or image-understanding test. These capabilities must retain separate acceptance labels.

Stop currently forwards interrupt operations to the handles opened for Main and the wired profiles. This establishes a source path, not complete proof that every queued admission, delayed notification, delegated task or tool subprocess is cancelled. The owner requested Stop All as an immediate product control. A bounded test should define and exercise the promised scope using Hermes's existing cancellation owners, without adding an application scheduler.

Approvals and clarifications are also distinct from a generic chat reply. Hermes's client has protocol support, but the current app must demonstrate that a real pending approval or question is rendered, answered by the correct user, and correlated to the correct profile. A text response containing the words approval needed is not a structured approval workflow.

These acceptance gaps should not all block a chat-only private preview. They do block advertising the corresponding capabilities as complete. The release feature list must say which controls are included, tested, unsupported or pending.

19 Runtime conclusion and the boundary to freeze

The ordinary local Main and direct-agent conversation path is a useful foundation. The app does not need another generic runtime to move forward. Hermes owns the model/tool loop and Bot machinery; the application should preserve the small transport and presentation boundary that now works.

The current proof is narrower than all Bot Mode semantics. Direct @Name requests and profile-specific logs establish identity and execution for those requests. They do not establish the asynchronous Main-to-agent-to-Main foreground presentation. The latter deserves one exact acceptance scenario, not another broad refactor.

The next implementation change should be chosen from the waterfall plan and mapped to its existing owner. A successful stage should leave the previous working chat tests unchanged and show the new behavior through the real product. A failing stage should be diagnosed at that boundary before unrelated files or stores are removed.

The owner's exhaustion is grounded in repeated replacement and restoration cycles documented in the Git record and conversation. The engineering response should be shorter, verifiable increments and honest status, not another assurance that a large rewrite will finally solve everything. The report's proposals are intentionally separated from the evidence so that they cannot silently become the next unapproved runtime.

20 Hermes Desktop dashboard and Bot Mode

The report distinguishes three things often called a Hermes UI: the installed official Desktop application, the installed official browser dashboard, and independent community web applications. They do not have interchangeable implementations. The installed source is the authority for LiquidAIty's actual integration; current upstream documentation is comparative evidence, not proof that this checkout was updated.

Official Desktop

Hermes Desktop uses the existing agent configuration, profiles, tools and session state. Its gateway client provides transport, profile routing, attachment tracking, request budgets and protocol handling. The client conditionally attaches a conversation on its socket before an operation that requires attachment. It records an attached snapshot so the same operation need not blindly attach every time. [R22 E01]

The Bot UI identifies a profile's titled Bot Chat. On explicit navigation to a Bot, Desktop can perform a lookup and force a fresh resume for viewer hydration. These are view-opening actions, not evidence that every user message should trigger an application-owned lookup/resume/history sequence. The distinction matters because earlier work confused the costs of opening a viewer, delivering work to a profile, and assembling the model's conversation history. [R23]

The application currently reuses HermesGateway from that Desktop code. That is evidence of reuse, not proof of a stable third-party public SDK contract. Future updates need exact compatibility checks around the imported client and gateway ticket code. The current working import should be pinned and preserved during feature work rather than replaced with a new protocol after every upstream change.

Official browser dashboard

The installed web directory is a Vite/React dashboard for configuration, keys and session monitoring. Its ChatPage embeds the Hermes terminal UI through xterm and a PTY WebSocket; it does not simply duplicate the Desktop React chat implementation. Its Sessions page loads transcript messages when a session is expanded and can navigate into chat using the stored session. [R31 R32]

This yields a useful lesson: a UI can expose Hermes execution without becoming the agent runtime. It does not mean LiquidAIty should replace its accepted Main chat and Cards with a terminal. The owner wants a different product surface, with the same underlying execution owner.

Bot to Bot delivery

The installed Bot path is message_agent through Bot live delivery into the recipient's authority. The authority resolves that recipient profile's Bot Chat and can create it when absent. An open chat viewer is not a prerequisite for delivery. The human composer prompt.submit interface and internal automation bot_relay.deliver interface are not interchangeable names for the same user operation. [R24 R25]

The Bot implementation contains its own receipt and polling behavior in places. The report does not recommend removing Hermes's internal mechanisms because the application should avoid redundant roster or reply loops. A legitimate dependency timer and an unnecessary application pre-send wait are different findings and require different evidence.

Hermes Projects

Hermes has a real per-profile projects.db containing named Projects, folders, primary paths, an active Project pointer, optional board association and repository cache. Workspace tools can change the calling GUI session's working directory without moving the global selection for other chats. These are useful existing capabilities for workspace organization and Builder binding. [R34]

However, Bot recipient resolution still calls get_session_by_title for Bot Chat inside the recipient authority. The Project store does not automatically add LiquidAIty's authenticated Project identity to that lookup. A workspace folder is not equivalent to a user-authorized application Project or its graph workspace. Reuse what Hermes Projects actually provides; do not describe it as completed multi-tenant isolation.

21 Open WebUI comparison

Audited upstream snapshot: f7ac7ee10962ddec0017f5c29b6740d9b9c5faca, dated 29 October 2026 UTC. This is not a version installed in LiquidAIty. The comparison reviewed model/chat/user-memory schemas, context compaction, persistence and scaling guidance.

Open WebUI separates reusable model/prompt presets from user-owned chats. Its memory records have a user identity and owner checks. That is the useful design principle: one reusable definition does not mean one shared active transcript. LiquidAIty should preserve that separation through its existing profile and Project owners rather than import another schema. [E02a E02b E02c]

Its ordinary default forwards full conversation history. Optional compaction introduces summary checkpoints and retains recent messages; original history remains available for display. The inspected implementation chooses safe user/tool boundaries and separates system instructions. The default is not evidence that full replay is universally desirable. It conflicts with the owner's selected last-three-pairs direction. [E02d E02e]

Documentation and source did not perfectly agree about the default compaction model. The pinned implementation uses the explicit compaction model when configured, otherwise the chat model. This is a concrete example of why a report must resolve claims against source instead of copying documentation that sounds convenient.

Its persistence supports SQLite and PostgreSQL. Its scaling guide distinguishes single-instance local deployment from replicas requiring shared coordination and storage. Distributed cancellation is routed to the process owning the task. That is a useful execution-owner principle, but copying its task manager into a Hermes UI would create the duplicate ownership the owner rejected. [E02f E02g]

Adopt as ideas: transparent active-input usage, durable transcript separate from replay, honest deployment envelopes, and bounded rendering/database work during streaming. **Do not adopt automatically:** full-history defaults, its alternate task infrastructure, or third-party context filters as a substitute for understanding the Hermes seam. Its license also has branding-related conditions; copying code requires deliberate license review, not an assumption that every open-source project is MIT. [E02h]

22 LibreChat comparison

Audited application snapshot: f4154b649efa37e1345ccc517a4d8d1ed2178c82, dated 31 October 2026 UTC. Audited agents-library snapshot: 708c9118f478a84f28e8e253148a373b49fec191. The review covered agent and conversation schemas, context retention, memory extraction, RAG, steering and resumable streams.

LibreChat keeps agent definitions separate from user/conversation/Project records. Its recent-turn retention is particularly relevant: a retained user-led turn includes the following assistant and tool messages through the next human-authored message. This protects tool integrity better than slicing arbitrary rows. Its retention option affects compaction; it should not be mislabeled as a permanent last-N-only policy. [E03a E03b E03c]

The agents library uses pressure-aware handling of already-consumed tool results before more expensive compaction or truncation. Current outputs that the assistant has not yet used are treated differently from old consumed observations. LiquidAIty can use this as an acceptance principle without transplanting LibreChat's message engine. [E03d]

LibreChat's memory.messageWindowSize controls the recent messages reviewed by an optional memory-extraction agent. It is not the number of messages sent to the main answer model. This distinction directly supports separate, explicit UI labels for history, retrieval and background learning. A single Memory 5 control would be misleading. [E03e]

The inspected file-search path retrieves five candidates per file, combines results and retains ten overall. It scopes user attachments differently from shared agent resources. These are concrete defaults for that path, not a universal ideal for LiquidAIty. Retrieval count, history count and memory-extractor count remain independent. [E03f]

LibreChat defines queue, steer and interrupt behavior and checks the executing run's identity. It can detach generation from a browser connection using resumable streams; multi-instance coordination uses Redis. Its documentation explicitly distinguishes surviving a client disconnect from surviving a generating server failure. The useful lesson is precise guarantees. The wrong lesson would be to add its job manager, stream store or queue to LiquidAIty when Hermes already owns those responsibilities. [E03g E03h]

Adopt as ideas: complete-turn retention, truthful admission controls, exact execution attribution and authorized retrieval. **Do not adopt automatically:** another generation engine, a memory-extraction model on every request, or a new RAG service that duplicates Engraphis.

23 AnythingLLM comparison

Audited snapshot: 9caf2b4c29d9782f1cba4142be2525ebfa0ecd43, dated 30 October 2026 UTC. This comparator provides a direct example of a genuine recent-exchange window.

The workspace openAiHistory setting defaults to twenty. The inspected query selects recent workspace/user/thread/API-session rows, each containing a prompt and response, then orders them chronologically for input. Older rows remain stored. Thus bounded recent conversation is a normal product choice, not an impossible request. Copying this JavaScript path into Main's route would still be the wrong implementation boundary for this Hermes-based application. [E04a E04b]

Retrieval topN defaults separately to four. Workspace knowledge and thread attachments have different scope. The result set and the recent conversation window are assembled as distinct inputs. This is a strong UI and data-model lesson for the proposed three recent pairs and bounded retrieved source policy. [E04c E04d]

Its optional memory feature separates global per-user memories from per-user workspace memories and performs background extraction under its own conditions. That is not equivalent to a foregound context compaction setting. LiquidAIty should not copy another memory extractor or a relevance-to-recency fallback: the owner's Relevant and Recent distinction is explicit. [E04e]

AnythingLLM uses SQLite by default, with separate workspace, user, thread and chat records. Its desktop distribution is single-user, while the server distribution supports multi-user administration. Enabling access to a desktop instance does not establish the server product's isolation. This is a useful precedent for LiquidAIty's staged launch envelope. [E04f]

There are also deliberate differences not to copy. The inspected streaming path saves a pending prompt before generation, whereas the LiquidAIty owner selected completed-pair application persistence after reply. Its client-disconnect cancellation policy differs from LibreChat's durable generation choice. These are product tradeoffs, not proof that one universal architecture must be imposed on LiquidAIty. [E04g]

24 Letta comparison

Audited current harness snapshot: letta-ai/letta-code at 3b7e3ffcf16d6ba4880f7a697261837000221320, dated 2 November 2026 UTC. The current letta-ai/letta repository redirects readers to that harness. Older server architecture should not be presented as the inspected current implementation. [E05a]

Letta distinguishes an agent's identity and durable memory from its independent conversations. Local records include agent_id, conversation_id and in_context_message_ids. A conversation therefore has its own active working set without requiring a different reusable agent definition. This directly supports the need to separate LiquidAIty's profile identity from Project conversation state. [E05b]

Its transcript reader and active-context state are separate projections. Compaction changes selected message IDs rather than pretending original history no longer exists. Local transcript search and cloud search have different capabilities. Its MemFS is a Git-backed memory filesystem, not a vector database by default. That is a useful distinction, not a recommendation to replace Engraphis with files. [E05c E05d]

Letta's memory-learning or dreaming work is distinct from compaction. It consumes additional model work and can update durable memory after completed activity. The relevant lesson for ThinkGraph is to separate learning from foreground reply timing. The existence of another product's automatic learning does not authorize a new extraction scheduler here. [E05e]

The App Server assigns turn execution, tool preparation, queueing and events to one runtime while clients own product state. This is the same ownership principle LiquidAIty should preserve with Hermes. Installing a

second runtime alongside Hermes to obtain this principle would defeat the point. [E05f]

Adopt as ideas: explicit independent conversation state, original history recovery, active-context inspection and runtime-owned event/control semantics. **Do not adopt automatically:** cross-Project shared agent memory, MemFS, another agent server, or background learning whose privacy/cost policy the owner has not chosen.

25 Dify comparison

Audited snapshot: ca09f448484d5945fe9852968df903dba2c1cc29, dated 2 November 2026 UTC. The source review focused on Chatflow/classic conversation and retrieval paths, not every recently added Agent subsystem.

Dify's history memory reads stored query/answer rows, chooses the current branch and bounded history, skips an unfinished current answer, then constructs prompt messages. It deliberately avoids reloading a stored fully expanded prompt as historical content, because doing so can recursively include earlier history and grow input quadratically. This is a concrete audit principle worth preserving in any system. [E06a]

Node-level memory is scoped to the current conversation. Retrieval Top K and reranking are separate stages. The inspected retrieval default used four candidates with optional reranking and thresholds disabled. These are path-specific implementation facts; they do not establish that four is best for the user's work. [E06b E06c]

Its integrating backend must supply a correctly authorized end-user identity. Conversation scope combines app, tenant and end-user identity; a client-provided identifier is not authentication by itself. LiquidAIty's authenticated Project access and profile owner checks should be preserved and extended coherently for its chosen deployment, not replaced by trusting a Project UUID in model text. [E06d]

Dify is a much larger multi-service platform with storage, coordination, vector, sandbox and network-protection components. That architecture solves its product requirements, but is not an implementation backlog for LiquidAIty. Its source license has additional multi-tenant and branding conditions, so borrowing code or embedding it would need an explicit license decision. [E06e E06f]

Adopt as ideas: explicit app/user/conversation scoping, bounded historical projection, separate retrieval stages and protection against recursively stored prompts. **Do not adopt automatically:** a new workflow engine, classification router, vector service, or platform deployment that replaces the current Hermes owner.

26 Community Hermes interfaces and the transplant risk

Audited community Hermes WebUI snapshot: eccc701b9acc73559d76a1baafc6142fd69d9dd6, dated 1 November 2026 UTC. This is nesquena/hermes-webui, not the installed official Desktop or dashboard.

Its ordinary chat path runs Hermes in-process and passes projected history to agent.run_conversation. It owns additional stream registries, cancellation flags and agent instances. Its WebUI session JSON remains a transcript authority in the inspected architecture. A proposed unification adapter is documented separately and must not be described as the active production replacement. [E07a E07b E07c]

The project's own source-boundary RFC inventories private Hermes imports and direct state-schema coupling. Its runtime-adapter documentation says the default remains the earlier direct route while later wiring is separate. These are strong reasons not to transplant its runtime/session code into LiquidAIty simply because both products display Hermes chat. [E07d]

One implementation addresses profile-memory binding races caused by process-global Hermes-home changes. This is not proof of that exact bug in LiquidAIty. It is evidence that concurrent profile scoping includes memory providers and credentials, not just profile names or conversation IDs. [E07e]

Useful visual ideas include profile attribution, context-usage display, clear compatibility information and real activity. Its architecture and authentication defaults are not a demonstrated hosted multi-tenant service. A public claim requires the actual deployment and threat-model evidence, not a screenshot of a login form.

Other ecosystem web/workbench projects can be studied as delivery and UI examples. Their own servers, databases, adapters and updater behavior must be disclosed. A catalog entry or a project name containing Hermes does not establish that it uses only public extension surfaces or is appropriate for this app's preservation constraints.

27 Hermes plugin catalog and ecosystem positioning

The installed catalog contains **762 unique named entries** across 763 YAML files; removed.yaml is a special blocklist rather than an installable entry. All 762 entries have full forty-character hexadecimal commit pins. The declared tiers are 741 community, 17 official and 4 verified. Compatibility requirements appear in 484 entries, capability declarations in 760, and nonempty known-issues fields in 73. These are local metadata observations, not remote execution or security acceptance. [R33]

Category using the catalog default	Entries	Interpretation
Tools	247	Declared tool integrations
Desktop	176	Includes two entries whose missing category defaults to Desktop
Memory	67	Alternative memory and recall policies
Models	66	Provider and model integrations
General	53	Other declared integrations
Automation	49	Background or workflow integrations
Web	42	Web and browser related integrations
Platform	41	Messaging or platform integrations
Voice	21	Audio and voice integrations

The CSV appendix contains every named entry, declared category, tier, repository, pin, compatibility declaration, capability lists, a short description excerpt and source hash. It records review depth explicitly. Excerpts are labeled and do not replace the full pinned declaration. Two raw category fields are absent; the table above applies the catalog's documented Desktop default. An exact pin improves reproducibility, but does not prove that every remote commit is still available or that a plugin suits this application's scope.

Available installed and enabled are different

The root general-plugin directory was empty and the root Desktop-plugin directory absent. Builder's plugin/hook directories were empty. Main, ThinkGraph and KnowGraph explicitly disabled project-graph and had no enabled entries in the inspected configuration. Disabled project-graph residue remains in those profiles; the inspected Main directory contained only compiled cache residue. The report does not claim that every trace was removed.

Bundled providers, context engines, memory providers, general plugins, event-hook folders, Desktop plugins and dashboard plugins have different activation rules. An empty general allow-list does not prove that every built-in tool/provider is disabled. Do not use one plugin list to make claims about all of those surfaces.

High relevance entries

Holographic is a separate memory provider with its own SQLite fact store, search and optional embedding behavior. It is not Main's state.db and should not be confused with Engraphis. LCM-X and Chronicle are alternative continuity/memory architectures with additional stores and retrieval surfaces. Honcho is an explicit personal-memory and hosting/privacy choice. Cognee and qdrant-memory overlap existing graph/memory ownership. None is necessary merely to keep today's working Bot Chat path. [R33]

Context-audit, context-overview and token-speed are useful observability references. They do not make inference faster. Langfuse or OpenTelemetry may be appropriate later if an observability owner and privacy policy are chosen, but adding multiple exporters or full-content capture by default would increase risk. Tool-slimmer should be considered only after measured tool-schema cost is material and only through supported public surfaces.

Workbench and Talaria are useful UI/distribution comparisons. Their separate backend extensions and private-source dependencies must be understood. Project-stewardship, prompt-queue and other task/project owners conflict with this app if installed merely because their names sound relevant. The owner explicitly rejected another prompt queue. Snapcompact's image-based representation is not a necessary answer to three recent pairs and exact text retrieval.

The full catalog has useful options, but it is not a shopping list or a launch dependency list. The report recommends no plugin installation. Choose one only when the existing owner cannot meet a specific approved need and its data flow, scope, cost and compatibility have been reviewed.

Becoming a credible Hermes ecosystem application

LiquidAIty does not need to become a loadable plugin to be a Hermes-based application. Its strongest proposed identity is a Project workbench with visible agent topology and inspectable reasoning/evidence, while Hermes remains execution owner. A small Hermes-side connector may eventually be justified for an exact public extension seam; it should not be used as a label for uploading the entire application and its runtime assumptions.

For catalog publication, the inspected rules require a public loadable repository, truthful manifest and capabilities, a full commit pin, an owner/maintainer submission, validation, disclosed networking/storage/background work, supported extension APIs, proper profile-secret access and maintainer review. Updates require reviewed pin changes. Desktop extensions must stay within the SDK rather than patching internal stores or DOM. The tier is not self-awarded. [R33 E15]

The recommended order is correctness, local/private acceptance, clear release documentation, then optional ecosystem publication. Catalog admission is not a substitute for Project isolation or a requirement to continue useful development.

28 Code removal rejected work and cost of churn

The owner's account of repeated deletion and restart is supported by concrete repository evidence. It cannot be reduced to a precise count of all wasted work because discarded uncommitted changes are not completely represented in Git. The report therefore distinguishes committed line-change events, endpoint snapshot differences, reset events and explicit removal/restoration checkpoints.

Measure	October 1 through audited HEAD	September 1 through audited HEAD
First party source insertion events	246950	365406
First party source deletion events	275846	341711
First party source file change events	4186	5912
First parent commits with counted text changes	85	169

For this calculation, first party source means source-like extensions under apps, client, packages, services, scripts and prisma, excluding node_modules, dist and virtual environments. Extensions counted were ts, tsx, js, jsx, mjs, cjs, py, ps1, sh, sql, prisma, css, scss and html. This is a transparent structural scope, not a semantic judgment that every line was application-authored or wasted.

A deletion event counts a line removed in a commit diff. The same logical line can be added, changed, removed and reintroduced repeatedly. Reformatting and rewrites also contribute. These totals must not be described as unique lines of useless code. They do demonstrate substantial change volume and repeated work.

An independent endpoint comparison from the last first-parent commit before October 1, 6e07220353a2d5db9a5c25aecfba4093ef4e2607, to the audited HEAD across the same six top-level directories reported 554 files changed, 67169 insertions and 95970 deletions. This broader directory snapshot includes text files beyond the extension-filtered source metric. Its net reduction is 28801 lines. It is not the same statistic as cumulative churn.

The available HEAD reflog since October 1 contained 107 entries, including **18 reset entries**, the earliest retained qualifying reset on October 12 and latest on November 1. Reflog text does not establish that each was a hard reset or discarded a distinct rejected implementation. Git restore and uncommitted deletion may leave no comparable HEAD event. Thus eighteen is a count of recorded reset events, not the exact number of times the owner rejected all changes.

Selected removal and restoration checkpoints

Commit	Recorded subject	What it establishes
68b2014e7	Remove app owned runtime for a clean thin rebuild on updated Hermes	A deliberate runtime removal checkpoint; counted first party source deletions 23237
8a8f550dc	remove TypeScript fake bot runtimes	Another removal checkpoint; counted first party source deletions 15952
3300e6142	Restore real Bot Mode	A restoration marker not standalone runtime proof
363a6170f	Restore pre refactor UI baseline from 6530ff62f	UI restoration was separately necessary
3ccf4cd7d	remove failed Main messaging implementation preserve Card startup and CLI	Explicit rejection of a Main implementation while preserving useful owners
a1e4cc7bd	remove poisoned runtime and routes before Main refactor	Further runtime removal rather than simple feature extension
7e7647656	remove rejected observer paths and align Card sessions	Rejected observer behavior was separately removed
6495f3017	remove EventSource paths and reduce Bot Mode seam	Another boundary reduction before proof
338a7d5fa	Restore working Main and ThinkGraph flow after failed plugin rollback	Plugin rollback itself required recovery
5c734cf4f	removal of rejected research machinery and custom map plugin	Rejected research and view integration reached committed cleanup
d36fb4db4	Remove ThinkGraph from Main	Separation of Main from Think processing restored in source
05285ac62	Checkpoint Bot Mode working reasonably well	Current tested baseline not a claim that every feature is complete

Whole-repository totals are far larger because they include imported trees, generated text and a multi-million-line old Hermes removal. For example, commit 293a2330d removed about 3.4 million text lines while removing the old Hermes tree. Presenting that number as assistant-written application code would be misleading. The report deliberately reports the narrower first-party-source scope alongside the classification rules.

What can and cannot be attributed

Git author fields often identify the person or process that committed, not who generated each line or approved each design. The supplied conversation provides the owner's account of repeated scope violations and false success claims. The commits and reflog corroborate repeated churn and recovery; they do not independently quantify every incident, dollar spent or lost opportunity. No monetary loss estimate is invented here.

The actionable diagnosis is a sequence of engineering failures: familiar architecture substituted for the requested boundary; proposals treated as authorization; duplicated owners hidden behind plausible names; limited proof promoted into product acceptance; and later changes built on checkpoints whose limitations were not explicit. The unusual product design did not make those failures necessary.

29 Feature and idea preservation audit

Code removal and capability removal must be assessed separately. A discarded implementation may have been wrong while the user-facing idea remained valuable. Conversely, a removed wrapper may have added no legitimate capability. The following ledger identifies the currently supportable classification without automatically restoring anything.

Capability or idea	Current classification	Required treatment
Main and direct addressed chat	Observed working baseline	Preserve identities and existing Hermes execution
Builder CLI	Retained source path with historical live evidence	Keep the actual Builder terminal; separate fresh acceptance from chat
Voice forwarding	Retained route source; fresh full voice test not run	Do not remove as adjacent runtime residue
Image forwarding	Retained route source and historical proof	Include fresh acceptance if release scope promises it
Completed pair storage	Retained and focused tests passed	Keep after reply ordering and original source
Source chunks and citations	Retained source and stored index	Preserve exact links; prove full desired coverage
Research requested flag	Retained product requirement	Flagging is not automatic research dispatch permission
Storage to Think delivery	Desired capability currently disconnected	Complete through existing storage/extraction owners after approval
Visible graph map per turn	Desired view policy; ordinary delivery unproven	Do not restore rejected map plugin or invent map tools
Relevant and Recent distinction	Explicit retained requirement	Do not replace failed relevance with recency
Card model prompt reasoning controls	Saved fields exist; effective Bot parity unverified	Prove save to actual profile to real next turn
Live graph refresh after writes	Incomplete source wiring	Complete existing committed-change boundary without polling
Asynchronous Main delegation result	Hermes machinery exists; foreground later reply unproven	One exact acceptance test then smallest existing view seam
Project scoped conversations	Required but not established by current fixed Bot Chat lookup	Isolation stage before multi-Project claims
Data Anchors in ordinary Bot chat	Nonempty selection currently rejected explicitly	Preserve honest refusal; separate approved integration decision
Run and IDF inspection	Separate retained path not proven by Bot chat	Do not claim it happened because a message replied
Team and Magnetic task execution	Deferred in current plan; direct chat works	Do not promote greeting tests into worker execution proof

Capability or idea	Current classification	Required treatment
Trading execution	Intentionally paper only and unapproved	Separate domain authorization and acceptance
General NRGSIM provisioning	Bounded local implementation only	Preserve accepted local modeler; separate generalization work
Activity and agent status	Useful product idea; old observer implementations removed	Reuse real runtime events only when scope explicitly includes it

This is not an exhaustive reconstruction of every idea ever discussed. Historical reports, removed files and owner recollection can identify further candidates, but they cannot automatically authorize restoring old code. A future preservation pass should add evidence for a specific capability and classify it before changing anything.

How to prevent the next accidental loss

Before an edit, state Change, Keep and Owner/path in the task. Before a deletion, list affected capabilities and inspect actual callers. After the edit, verify the previous working path as well as the new result. If a useful control is removed as collateral damage, that is a regression, not acceptable cleanup. If an abandoned path remains live after replacement, the replacement is incomplete.

The new repository skill encodes these distinctions. It is not an automatic guarantee. The read-only planning evaluation covered redundant session removal, research-only memory comparison, runtime removal while retaining voice/CLI, and three-pair input with full durable history. It passed those planning cases, but no future implementation compliance is claimed in advance.

30 Known work versus owner decisions

Known work is evidence-backed missing behavior under an already selected requirement. A decision changes architecture, product scope, cost, privacy or user experience. Mixing these categories makes the owner repeatedly answer settled questions while unapproved implementation slips in elsewhere.

Already selected or established	Still requires an owner decision	Not authorized merely by this report
Card is UI for the Hermes profile	Exact migration/reconciliation of existing saved settings	A second profile configuration owner
Three recent complete pairs is the initial history direction	Retrieval timing and strict selection failure behavior	A new context engine or provider modification
Full ideas and original source remain recoverable	Retention archive export and account deletion promises	Clearing databases or replaying old data
Engraphis and Graphiti remain graph owners	Which learning may cross Projects within one user	Another memory store or graph writer
No automatic KnowGraph research now	Future research dispatch and flag-consumption semantics	Research queue or request registry
Working chat is preserved	Local first launch versus hosted first launch	General runtime replacement

Already selected or established	Still requires an owner decision	Not authorized merely by this report
Wires determine authorized reachability	Supported per-conversation/project authority mechanism	Global roster relaxation to make tests pass
Current reported times miss the Main target	Six-second first useful text versus complete response SLO	Model switch or prompt truncation without approval
Exact current launch and saved state preserved	Supported OS distribution and upgrade strategy	Hermes update installation or deployment
Report includes ecosystem positioning	Standalone application versus optional loadable connector	Catalog submission or publication

The best next decision is the first launch envelope. A local single-owner preview and a hosted multi-user service have materially different exit gates. Choosing one does not delete the longer-term ambition. It prevents work on ten-million-Project capacity from consuming the budget needed to prove a useful first product.

Decisions should be resolved at the relevant stage

Do not demand every future decision before fixing one already-known gap. For example, proving a Card's model setting reaches Hermes does not require choosing a cloud database. Connecting exact stored pairs to Think extraction does not require automatic research policy. Conversely, launching untrusted users cannot defer the user/Project isolation decision as an edge case.

Each waterfall stage below names its entry condition, known work, decision boundary, proof and stop condition. The report is not an instruction to execute all stages now.

31 Waterfall stage zero and profile truth

W0 Preserve the recovered baseline

Objective: make the current checkpoint a dependable foundation rather than restart runtime design. **Status:** current ordinary Main/direct-agent live evidence, builds and eight focused tests exist; the complete release suite does not.

Preserve current source, user data, profile identities, saved wires, the accepted UI, voice/image entrances and Builder CLI. Record the exact code/configuration and existing full launcher. Do not perform another broad restore, dependency update or runtime deletion as preparation. Git operations remain owner-authorized separately.

Exit evidence: current baseline report, repeatable natural-message test steps, source identity, measured results and explicit unproven capabilities. **Decision:** what the first release promises. **Stop condition:** a proposal introduces another execution owner or cannot name the working behavior it preserves.

W1 Make profile controls truthful

Prerequisite: W0. **Objective:** every included Card Runtime control reflects and changes the actual Hermes profile used for ordinary chat.

Known work: map each field from UI save through its existing persistence/transport owner to effective Hermes readback. Toolsets/skills already have a direct profile API path. General model/prompt/reasoning parity is not established on the ordinary path. Start with one reversible field and one profile, then extend only after the path is proven.

Decision: how existing saved revisions represent profile settings without becoming a competing authority. **Preservation:** other profiles, grants, credentials, topology, saved Cards and ordinary send timing. **Acceptance:** save through UI, read the same profile, run a real next turn, verify effective settings and another profile unchanged. **Failure response:** diagnose the field's existing owner; do not add per-message publication or a polling revision tracker.

Deliverable: a small reviewed implementation and field-level acceptance matrix, plus targeted canonical documentation if that change is authorized. **Not required:** changing history selection, graph automation, cloud hosting or all profile tools at the same time.

32 Waterfall history and interaction stages

W2 Implement and evaluate the selected history policy

Prerequisite: W0 and truthful configuration ownership for any settings exposed in W1. **Objective:** ordinary model input includes the selected three recent complete exchanges and relevant authorized memory without replaying the entire conversation.

Known work: inspect Hermes's supported request-context seam; retain valid current tool state; preserve original stored history; verify source coverage; inspect the actual provider-bound input. An existing optional hook is not the same as a configured working policy. pre_llm_call injection must not be mistaken for history replacement.

Decisions: automatic bounded retrieval on ordinary turns versus agent-requested retrieval; treatment of a compact prior summary; strict failure behavior when selection/retrieval fails; handling of a genuinely large current task. Do not silently decide that a missing retrieval means empty memory, recent-as-relevant, or full-book replay.

Acceptance: a direct follow-up resolves the prior alternatives; a correction remains controlling; an older idea is retrieved with exact source; unrelated Project information is excluded; the full stored transcript remains recoverable; tool-call/result structure remains valid; provider input and measured latency match the selected behavior. Compare like-for-like outputs and reasoning settings instead of attributing every random latency change to the edit.

Stop condition: the implementation creates another session manager, transcript ledger, graph writer or generic retriever when an existing owner can do the work. A necessary extension or vendor change requires an explicit proposal and approval.

W3 Prove continuous interaction and later replies

Prerequisite: W0; can use the existing history policy while W2 is separately reviewed. **Objective:** user input remains usable and real subsequent results appear under correct identity.

Known work: exercise existing Hermes steer/admission behavior, a real Main message_agent call, delayed recipient completion, foreground return, Stop All, and structured approval/clarification if included. This is not an invitation to invent a prompt queue or reply poller.

Acceptance: input is accepted or honestly rejected only for a concrete loading/authorization failure; no message is attributed to another profile; later replies are visible; interruption reaches the declared scope; no fabricated status replaces execution. **Decision:** the exact stop promise for queued, active and child work. **Exit:** a small, repeatable product acceptance set tied to Hermes events and actual transcript records.

33 Waterfall graph and memory completion

W4 Complete new source to Think processing

Prerequisite: working completed-pair storage and exact source identities. **Objective:** a new saved exchange feeds the retained extraction and settlement owners without Main controlling Think processing.

Known work: connect only the approved storage-side entrance, preserve chunk snapshots and source citations, and prove Entity/relation/Think writes. Do not restore the rejected receipt search, usage ledger, generic target runner or main-completion wrapper. Do not migrate old tests merely to create apparent activity.

Acceptance: one natural new user exchange is stored; the exact pair and chunks are read; ThinkGraph performs real extraction; the expected records are materially persisted and read back; each Think can open its original source; irrelevant mentions do not become copied Thinks. Include a controlled failure with honest status and no fabricated pair identity.

W5 Complete Know research and committed graph refresh

Prerequisite: existing graph providers and authorized profile grants. **Objective:** deliberate research produces inspectable sourced Knows and real committed changes become visible in the correct Project.

Known work: prove an ordinary Bot research request, existing grounded evidence workflow, queue acceptance, actual persistence, exact citation readback and existing graph notification. Preserve separate observation and interpretation. A research flag remains data only.

Decisions: any future automation is separate; whether a consumed Think research flag remains set until a Know explicitly references it needs an exact owner-approved rule. **Acceptance:** correct source and Entity association, no duplicate writer, no queue receipt presented as completion, one correct refresh after commit, no other-Project refresh. **Stop condition:** a completion waiter, polling loop or research registry appears without an explicit need and approval.

W6 Prove the visible view supplied to agents

Objective: if graph-view context is in the release, the agent receives the intended compact visible Entity/Relationship map and selected evidence, not an invented extra graph-reading tool or hidden payload.

Acceptance: compare the user's current view and the actual model-facing data on the same turn; distinguish topology orientation from evidence; preserve top-result selection and source references; verify that graph failure is honest. This remains a scoped input/presentation integration, not another graph authority.

34 Waterfall isolation and release stages

W7 Prove Project and user isolation

Prerequisite: a chosen profile/runtime hosting model. **Objective:** reusable agents operate across Projects without sharing private conversations, credentials, roster or retrieval scope accidentally.

Known blockers: fixed profile Bot Chat lookup, profile-level Project credential/roster publication, and a global profile-owner namespace. **Decision:** which supported Hermes surfaces provide per-conversation authority, or which smallest reviewed extension is necessary. Do not weaken owner checks or clone every profile per message as a shortcut.

Acceptance: two Projects with different wires and private content, then two users; simultaneous requests; actual recipient, tool principal, working directory and retrieved source checks; no leakage and no false success.

A workspace path alone is not the Project authorization boundary.

W8 Ship an honestly scoped local or private preview

Prerequisite: explicit release scope and acceptance of the included W1-W6 capabilities; W7 is required if multiple independent Projects/users are promised. **Objective:** another intended user can install and operate the promised product without the developer explaining hidden prerequisites.

Known work: supported environment, full launch, credential onboarding, clear errors, durable data reuse, current regression/build checks, export/recovery procedure and upgrade rehearsal. Include voice, images, graphs and domains only when their acceptance is complete or clearly disabled/unavailable in that release.

Exit: one supported clean machine and realistic product workflow. **Do not require:** every optional plugin, every feed, live trading, all NRGSIM systems or a cloud fleet merely to release a useful local product.

W9 Public hosted launch gate

Prerequisite: W7 and the chosen production architecture. Known work: protect internal services, abuse/rate/spend controls, identity recovery, privacy/retention/export/delete semantics, isolated tool/filesystem access, production assets, observability, backups and restore proof, load and cost tests, provider capacity and license review.

Acceptance: untrusted users cannot access each other's data or tool authority; remote clients cannot reach internal write services directly; operational failure and recovery are exercised; advertised latency/concurrency is backed by representative measurements. **Decision:** supported service envelope and operator responsibility. **Stop condition:** a local health check is used to claim SaaS readiness.

35 Waterfall optional expansion and ecosystem release

W10 Independent domain workstreams

Trading execution, generalized NRGSIM provisioning, broad WorldSignals feeds, advanced WorldView actions, automatic Team work and Magnetic task execution are independent feature projects. Each must use its retained owner, preserve current core chat and receive explicit acceptance criteria. A new domain should not force another generic runtime rewrite.

For Trading, execution requires explicit broker configuration, risk limits and owner authority; do not infer trading permission from a profile greeting. For NRGSIM, deterministic engine completion and engineering validity are separate. For live-world feeds, a healthy container does not mean all sources are fresh. For Team/Magnetic, worker selection, task completion and final synthesis require actual task evidence, not a connected wire or an assistant statement.

W11 Ecosystem publication

Prerequisite: a useful scoped release and truthful compatibility documentation. **Objective:** position LiquidAIty as a credible Hermes-based Project workbench, not a collection of unproven wrappers.

Known work: clear installation, screenshots of real behavior, reproducible acceptance steps, data ownership explanation, limitations, license inventory and supported Hermes compatibility. **Decision:** standalone application distribution, or a separately justified small loadable connector. A catalog connector must meet the actual catalog requirements; no publication is implied by this report.

Exit: an independent user can understand what the application adds, what Hermes owns, where data lives, what is tested and how to recover it. **Desirable later:** sample Projects with no private data, concise profile setup, optional telemetry chosen by the owner, and public compatibility tests. **Not a launch requirement:**

accumulating the largest plugin count or copying competitors' entire feature lists.

Sequence and budget discipline

The waterfall is deliberately dependency-driven. The owner should select one stage and its decision boundary. Complete it, prove it and preserve the previous baseline before opening another. Large estimates and calendar promises are omitted because the implementation choice, supported deployment and unknown upstream gaps have not all been resolved. A small approved discovery can estimate a specific stage; it cannot authorize the rest of the roadmap.

Funding risk makes this discipline more important. The highest value near-term outcome is a credible, useful release slice, not theoretical completion of every domain. A proposal is rejected if its complexity is not tied to an observable user benefit and a named existing owner.

36 Launch proposal register

Every item in this register is **proposed and not implemented by the audit**. Required means required for the stated release promise, not automatically required before any local use. Relative effort is deliberately qualitative: Small is a narrow existing-boundary change; Medium spans a few connected owners; Large includes a product or deployment decision. These are not delivery-time commitments.

P01 Truthful Card Runtime settings

Priority: release critical when editable agent settings are advertised. Owner: existing Card editor, profile API and saved configuration boundary. Proposal: reconcile one field at a time and prove the effective Hermes value and next real turn. Effort: Medium until authority is chosen. Risk: silent model/prompt override or competing stores. Preserve: every profile's identity, grants and other saved values. Exit: field-level UI-save, Hermes-readback and real-turn evidence. Approval: exact field authority and implementation scope.

P02 Project and account isolation

Priority: critical before multi-Project or hosted multi-user claims. Owner: existing authenticated Project/profile boundary and Hermes supported conversation/authority mechanisms. Proposal: isolate conversations, roster, credentials, tools, filesystem and retrieval together. Effort: Large because upstream capability must be established. Risk: privacy violation or a new duplicate runtime. Exit: concurrent two-Project and two-user negative tests. Approval: deployment/profile namespace and supported extension choice.

P03 Three complete recent exchanges

Priority: core product completion. Owner: Hermes request-context selection, not display history. Proposal: implement the owner-selected three-exchange working set while preserving full source and current tool structure. Effort: Medium, contingent on the chosen extension. Risk: malformed tool sequences, lost governing context or fail-open full replay. Exit: actual request inspection and continuity tests. Approval: selection failure policy and extension scope; the number three is already selected.

P04 Useful bounded recall

Priority: core memory promise. Owner: existing Engraphis retrieval/source indexing and existing graph context owners. Proposal: evaluate up to five returned passages from a broader candidate set, with exact source expansion and Project filters. Effort: Medium. Risk: missed evidence, unrelated memories or extra pre-answer model calls. Exit: retrieval evaluation for corrections, old ideas, exact identifiers and cross-Project isolation. Approval: automatic versus on-demand retrieval and proposed budgets; do not add another vector database.

P05 Storage owned Think processing

Priority: critical if automatic learning is advertised. Owner: retained conversation preparation/extraction/settlement. Proposal: connect new exact saved pairs through the accepted one-way path. Effort: Medium. Risk: repeating the rejected Main receipt/handoff architecture. Exit: natural new exchange to source-linked persisted Think with exact readback. Approval: one bounded stage; no historical replay or automatic research.

P06 Sourced Know acceptance

Priority: critical for sourced research claims. Owner: KnowGraph profile and Graphiti intake/readback. Proposal: prove one full deliberate research operation and exact citation/Entity persistence. Effort: Small to Medium depending on existing gaps. Risk: queue acceptance confused with persistence, observation mixed with interpretation. Exit: source passage, stored Know and visible inspector agree. Approval: test topic and write authority.

P07 Committed graph refresh

Priority: core graph UX. Owner: existing provider-write completion and Project notification/read hooks. Proposal: connect committed changes to the existing view refresh boundary. Effort: Medium. Risk: fake activity, cross-Project refresh or new polling. Exit: correct refresh only after real provider persistence. Approval: exact graph owner and event seam.

P08 Later Bot reply presentation

Priority: critical for Main orchestration claims. Owner: existing Hermes client events and Main view. Proposal: first run a real delayed delegation acceptance; if needed, use the existing event/timeline owner for later admissions. Effort: unknown until reproduced. Risk: another reply waiter, duplicate transcript or wrong profile attribution. Exit: actual Main call, recipient work, later correct foreground reply. Approval: test and any identified minimal repair.

P09 Continuous input and Stop All

Priority: core interaction requirement. Owner: Hermes admission/steering/interruption and existing UI controls. Proposal: define and test successive input, steering, active interruption and queued/child scope. Effort: Medium. Risk: UI lockout or claiming all work stopped when only one request stopped. Exit: observable controls match their labels and actual runtime state. Approval: scope of Stop All, not a new scheduler.

P10 Context and latency inspection

Priority: desirable near-term, useful for acceptance. Owner: existing Hermes usage/context events. Proposal: show input categories, first useful text time, final time and source references without streaming private prompt bodies to another telemetry store. Effort: Small to Medium. Risk: estimates presented as provider counts or observability mistaken for speed improvement. Exit: UI numbers match logs and label estimates. Approval: UI scope and any telemetry export.

P11 Reproducible local distribution

Priority: required before distributing a local product. Owner: existing launcher/build/deployment paths. Proposal: supported clean-machine installation, configuration onboarding, clear dependency readiness and controlled upgrades. Effort: Medium. Risk: automatic dependency mutation during startup or partial service launches. Exit: another supported machine runs the promised workflow and reopens durable data. Approval: supported operating systems and packaging format.

P12 Public identity and abuse controls

Priority: required before untrusted public access. Owner: existing authentication and deployment ingress. Proposal: choose one identity/recovery flow, rate and spend protections, session policy and operator controls. Effort: Medium to Large. Risk: a second identity store or exposing internal rails. Exit: authorized access succeeds, unauthorized scope fails, abuse limits and recovery work. Approval: public access and account model.

P13 Data lifecycle and privacy

Priority: required before deletion/export/privacy promises. Owner: each existing authoritative store plus Project lifecycle orchestration. Proposal: define archive, export, delete, retention and shared-resource semantics before implementation. Effort: Large if all domains are included. Risk: deleting shared/private data incorrectly or retaining data after claiming erasure. Exit: a disposable fixture inventory proves exact effects across stores. Approval: retention and recovery policy.

P14 Backup and recovery acceptance

Priority: required for a supported durable product. Owner: operator/deployment and existing databases. Proposal: document and rehearse consistent backup and restore under an explicitly authorized test. Effort: Medium. Risk: a file copy that omits WAL or external artifacts, or an untested restore promise. Exit: recovered Project state, conversation, graphs and files are usable. Approval: exact backup targets, location and recovery objective. No backup was created by this audit.

P15 Performance and cost envelope

Priority: required before latency or scale claims. Owner: existing model settings, measured execution path and deployment. Proposal: representative repeated tests with first useful text, completion, p50/p95, errors, tokens and provider cost. Effort: Medium. Risk: attributing one fast run to a change, or truncating quality to meet a metric. Exit: agreed SLO met for the defined workload. Approval: model/reasoning/output policy and test budget.

P16 Tool and filesystem isolation

Priority: required when untrusted users can run tools. Owner: saved grants, Hermes tool permissions and execution environment. Proposal: prove per-user/Project file and secret boundaries using the existing runtime's supported isolation. Effort: Large for hosted execution. Risk: a model prompt used as a security boundary. Exit: denied access remains denied even when explicitly requested by a model. Approval: execution environment and unattended permissions.

P17 Dependency and license release review

Priority: required for distribution. Owner: current lockfiles, controlled vendor register and release process. Proposal: inventory shipped code, modifications, licenses, notices and known advisories; select compatible updates separately. Effort: Medium. Risk: mass upgrades mixed into feature work or copying competitor code with incompatible terms. Exit: reproducible build and reviewed distribution obligations. Approval: any update or dependency change; this is not legal advice.

P18 Clear accessible product controls

Priority: required for the included user workflow; refinements can be staged. Owner: existing Card/chat/graph UI. Proposal: preserve accepted layout, clarify context versus history versus retrieval labels, keep keyboard/input behavior usable, and show real errors without fabricated success. Effort: Small to Medium. Risk: replacing familiar controls with explanatory panels or changing the accepted design during a runtime fix. Exit: owner-visible acceptance on the intended screens. Approval: exact UI change.

P19 Deliberate domain expansion

Priority: optional unless advertised. Owner: Trading, WorldView, WorldSignals, NRGSIM, Team and Magnetic respectively. Proposal: one bounded acceptance project for each selected domain. Effort: varies. Risk: domain scaffolding treated as real engine output or another generic runtime. Exit: domain-specific evidence, not a greeting. Approval: explicit domain scope and any external side effects.

P20 Hermes ecosystem publication

Priority: optional after a useful release. Owner: application distribution and any separately justified loadable connector. Proposal: publish truthful compatibility, installation, data-flow and acceptance information; submit a catalog component only if it meets the actual rules. Effort: Medium. Risk: another plugin owner, private API coupling or unreviewed self-updating code. Exit: independent installation and maintainer review if catalog inclusion is sought. Approval: publication and repository targets.

P21 Documentation truth and the repository skill

Priority: immediate maintenance, bounded to authorized documents. Owner: current canonical documentation and the new intent skill. Proposal: reconcile specific current/target contradictions and enforce Change, Keep, Owner/path before edits. Effort: Small. Risk: another enormous instruction framework that is not applied. Exit: no contradictory current owner claims in the affected boundary, and behavior demonstrated in subsequent scoped work. Approval: exact documentation reconciliation beyond the skill already requested.

P22 Preserve ideas while removing implementations

Priority: required for any future replacement. Owner: the active task's preservation set and existing source graph procedure. Proposal: classify affected capabilities before deletion, recover useful requirements without restoring wrong code, and compare old/new behavior through the actual user surface. Effort: proportional to change. Risk: useful voice, source citations, controls or identity checks disappear with a rejected wrapper. Exit: named preservation tests pass and no abandoned live path remains. Approval: the specific removal/replacement, not a broad cleanup license.

37 Review instructions and the next decision

This packet is intended for an independent GPT Web or Claude review as well as the owner. The reviewer should challenge the evidence and recommendations, not infer implementation permission from this document. Source paths refer to the inspected repository; external commits refer to research snapshots, not installed versions. A source-only finding should not be elevated to a reproduced incident.

Questions the independent reviewer should answer

Does the evidence justify preserving the current Main/direct-agent runtime and moving to bounded feature work? Which claim, if any, incorrectly treats direct addressing as proof of Main delegation? Does the report correctly separate the Card UI from the actual Hermes profile, and identify field parity without inventing another owner?

Does the proposed three-pair policy preserve the current task and complete tool sequences? Is the older-source retrieval plan faithful to Engraphis and Graphiti authority? Are the selection failure behavior and scope of shared learning explicit decisions rather than hidden assumptions? Is an optional context engine seam being mistaken for an already implemented feature?

Are the Project isolation blockers complete enough to cover conversation, roster, credentials, files and retrieval? Can a local release proceed without public multi-user readiness, provided the feature claims are honest? Which listed proposals are genuinely necessary for the selected first release and which should be

deferred to preserve funding?

Are deletion and reset statistics described accurately as events rather than unique wasted work? Does the feature ledger distinguish an intentionally removed implementation from a lost product requirement? Are comparator features treated as alternatives, with licenses and different assumptions, rather than imported as a mandatory architecture?

Why the process took so long

The repository and conversation show repeated scope expansion, duplicate runtime attempts, restoration and re-verification. There is genuine system complexity, but repeated rebuilding of a thin Bot integration was not an unavoidable consequence of the user's design. Some checkpoints were described more confidently than their evidence allowed, and subsequent work inherited those assumptions. Contradictory documents and changing source made later understanding harder rather than automatically better.

The report cannot diagnose a hidden training mechanism or claim this behavior occurs on every project. It can identify the observable errors and the controls needed to prevent recurrence. A new skill is useful only if the next edit actually follows it. The right proof is fewer unjustified changes, correct ownership, preserved features and accurate acceptance claims, not another assertion of understanding.

Whether to keep using Codex or switch

The owner owes no loyalty to a tool that consumes the project's budget without delivering reliable value. Switching to Claude, using it as an independent reviewer, or comparing both on the same bounded task are reasonable options. This report does not claim one model will necessarily perform better, and it does not invent comparative pricing or performance evidence.

The case for continuing here is narrow and testable: the current session has recovered working chat, produced profile-specific evidence, identified real gaps without changing production during the audit, and preserved a portable review packet. That becomes a reason to continue only if subsequent work stays within scope and reaches real acceptance with less rework. The owner's project and artifacts should remain portable to another engineer or assistant.

Recommended next owner decision

Choose the first release envelope and one next waterfall stage. My recommendation is to preserve the recovered runtime, prove Card-to-Hermes configuration parity, then implement the selected history/retrieval policy through an explicitly approved existing boundary. Do not start another general runtime rewrite. If the first release includes multiple independent Projects or users, schedule isolation as a mandatory gate before that release, not as an afterthought.

The report is not a promise that every remaining feature is small. It is a way to prevent all remaining work from becoming one undefined refactor. The owner should be able to accept or reject a stage by its observable result and preservation evidence.

Appendix A Repository evidence registry

All relative paths below are rooted at C:/Projects/LiquidAIty/main. Production source references describe the inspected 05285ac6227dfd438ba89f6180e669c9e3e1655a checkpoint unless the entry explicitly concerns the newly requested instruction skill. Line numbers are evidence anchors, not a claim that only those lines were relevant. No secret values, credentials or complete private conversation dump are included.

R01 Main transport

apps/backend/src/routes/mainSession.routes.ts. connectGateway imports Hermes Desktop client and local gateway support; prepareProfiles at 109-123 publishes MCP key and roster; /open at 153-163 opens profiles; /history reads application history; /chat at 180-247 submits and filters events; completion precedes appendConversationTurn; /stop and voice routes remain. Primary full source read.

R02 Main frontend owner

client/src/features/agentbuilder/console/useAgentBuilderMainChat.ts. Project-open effect runs history/open once for the Project; sessionsRef stores returned handles; requestMainText preserves per-request identity and active streams; onEvent handles text and completion; no post-response history refresh on the reviewed path. Primary full source read.

R03 Chat transport client

client/src/features/agentbuilder/console/mainSessionClient.ts. openProject, streamSession, SSE parsing, completion checks, stopSession, history loading and voice forwarding are distinct operations. Primary full source read.

R04 Saved Project bindings

apps/backend/src/services/projectCardAuthority.ts:57-88. Resolves enabled saved Card revision, actual profile, title and exact address, requiring one Main binding. Primary and source scout inspected callers.

R05 Profile ownership

apps/backend/src/services/projectStore.ts:209-232. assertCardProfileOwner queries distinct owners referencing a logical profile and rejects anything other than exactly the expected owner. This is a guard, not tenant provisioning.

R06 Direct profile controls

apps/backend/src/routes/cardProfile.routes.ts:53-105. Toolset, skill and learning operations forward to Hermes profile APIs. Source scout inspected the route and relevant tests; tests were not rerun in the reporting stage.

R07 General Card editor save

client/src/features/agentbuilder/state/useAgentBuilderCardEditor.ts:87-160. handleSaveSelectedCardConfig updates prompt/model/reasoning/options and persists the deck.
apps/backend/src/decks/deckDomainClient.ts:88-116 transports to Python.

R08 Saved Card domain

apps/python-models/app/card_domain_routes.py:138-151; apps/python-models/app/python_models/saved_cards.py:154-213,215-383,452-543,608-649; saved_card_update.py:47-214; saved_card_operation_validation.py:60-75. Source scout traced transaction, immutable identity, revision/grant writes, AGE projection and readback. No general profile publication observed in that reviewed path.

R09 Conversation source

apps/python-models/app/python_models/conversations.py:28-43,89-143,147-244,321-439. Full event scans, exact pair reads, chunk snapshots, indexing and semantic source search. apps/backend/src/conversations/store.ts transports application storage operations. test_conversations.py contains defined reopen, scope, rollback, pair and indexing tests; those Python tests were not rerun for this report.

R10 Per user Engraphis

apps/python-models/app/python_models/engraphis.py:32-80. Project owner is resolved from PostgreSQL, database path is db/users/<owner>/engraphis.sqlite, service cache is per owner, and SERVICE_LOCK is module-global. Source scout full boundary inspection and read-only database metadata supported the claim.

R11 Engraphis operations

apps/python-models/app/python_models/engraphis_operations.py:168-209,293-336. Bound service/Project operations and private conversation/source-search/preparation entrances. These are not automatically model-granted tools.

R12 Think graph route notifications

apps/backend/src/routes/thinkGraph.routes.ts:47-59,63-106. Source search route and existing Project event stream; only the route's retirement operation was found as a production notification caller in the reviewed first-party scope.

R13 Graph UI refresh

client/src/features/agentbuilder/state/useAgentBuilderKnowledgeGraphs.ts:227-321,480-491. Initial/confirmed-deletion loads and Think event subscription inspected. Know committed-change subscription was not established by the reviewed source.

R14 Graph preparation and settlement

apps/python-models/app/graph_routes.py:172-215 and python_models/thinkgraph_completed_pair_preparation.py and thinkgraph_completed_pair_settlement.py. Existing deterministic source/chunk and settlement owners; no ordinary save-triggered production caller established.

R15 Retained IDF path

apps/python-models/app/python_models/idf.py:27-100. materialize_idf owns the canonical Run input. Its existence does not prove ordinary Bot chat used a Run or IDF.

R16 Account session protections

apps/backend/src/auth/sessionStore.ts:18-117; auth/userService.ts:16-103; routes/auth.routes.ts:71-144. Source scout examined token generation/digests, expiry, cookies, passwords and current auth endpoints. No new account or sign-in test performed.

R17 Request and terminal boundaries

apps/backend/src/security/requestAccess.ts:56-108; routes/projectAccess.ts:27-43; routes/cardTerminal.routes.ts:42-93. Origin/socket checks, owner resolution and precise CLI identity. Existing security tests were reviewed as definitions.

R18 Server composition

apps/backend/src/main.ts and routes/index.ts:39-54; apps/python-models/app/main.py:42-54; services/knowgraph/app.py. Authenticated public feature routes and loopback/internal service boundary. Process liveness is not full provider acceptance.

R19 Startup and deployment

package.json:6-21; scripts/start-dev-services.ps1; scripts/start-dependent-services.ps1; docker-compose.yml; client/vite.config.ts. Existing full launcher, readiness and optional WorldSignals branch reviewed. No startup changes made for the report.

R20 NRGSIM

agent-products/nrgsim/services/routes.py:49-113; apps/backend/src/routes/nrgsim.routes.ts:15-54; agent-products/nrgsim/CoderReport.md. Current bounded local workspace restrictions source-confirmed; simulation receipts in CoderReport remain historical, with stale Main route references identified.

R21 Trading

apps/python-models/app/python_models/trading_contract.py:129-177; trading_tool_operations.py:92-200; trading_broker_observation.py:152-206; trading_jobs.py:391-442. Paper-only and execution-not-approved behavior source-confirmed. Trader greeting is not broker execution proof.

R22 Hermes Desktop gateway client

HermesLatest/apps/desktop/src/api/client.ts:405-516. Request routing, attachment tracking, conditional resume, snapshots and event handling. Read directly from installed source; not an app-owned replacement.

R23 Hermes Desktop Bot chat opening

HermesLatest/apps/desktop/src/plugins/hermes-bots/canonical-chat.ts:42-50,150-190,279-336,408-510,610-651. Profile/titled-chat identity, explicit viewer hydration and creation. Different from ordinary per-message duplication.

R24 Hermes Bot delivery

HermesLatest/tools/bot_mode_dm.py:208-219; tools/bot_live_delivery.py:32-75,468-506. Recipient owner resolution and relay call; some internal waiting/polling remains owned by Hermes.

R25 Hermes recipient authority

HermesLatest/gateway/session_bot.py:182-203,279-336. Resolves/creates the recipient's Bot Chat. No LiquidAIty Project-ID selection in that lookup.

R26 Hermes transcript storage

HermesLatest/gateway/session_transcript.py:631-656; HermesLatest/hermes_state.py:180-203. state.db is the profile transcript authority; active replay follows compression lineage.

R27 Active versus displayed messages

HermesLatest/hermes_state_messages.py:1340-1504. include_compacted is a display-read option; ordinary model replay excludes archived compaction rows. The full stored database is not necessarily the full active prompt.

R28 Request context selection

HermesLatest/agent/context_engine.py:121-152; agent/conversation_loop.py:1357-1399; website/docs/developer-guide/context-engine-plugin.md. Request-only selection, default no-op and fail-open behavior; injection is distinct.

R29 Compaction defaults

HermesLatest/hermes_cli/config_defaults.py:591-710; agent/context_compressor.py:2234-2255; website/docs/developer-guide/context-compression-and-caching.md. Absolute trigger, lean tail, protected messages and existing advanced settings.

R30 Micro compaction ordering

HermesLatest/agent/turn_finalizer.py:286-327,590-645; agent/micro_compaction.py; website/docs/developer-guide/micro-compaction.md. Post-turn work occurs before final persistence/closure, preserves user material, and can disrupt cache reuse.

R31 Official Hermes dashboard chat

HermesLatest/web/README.md; web/src/pages/ChatPage.tsx:1-125,1232. Dashboard embeds Hermes TUI through PTY/WebSocket and is not the same renderer as Desktop chat.

R32 Official Hermes session display

HermesLatest/web/src/pages/SessionsPage.tsx:391-475,741 onward. Expanded transcript display and navigation to chat; viewer work is not model input policy.

R33 Catalog and extension rules

HermesLatest/plugin-catalog/README.md:14-195; website/docs/developer-guide/plugins/catalog-submission.md; user-guide/features/plugins.md; developer-guide/desktop-plugin-sdk.md:23-32; application-declarations.md:1-25. All catalog YAML metadata was parsed. Twenty relevant entries were reviewed at declaration level; all remote implementations were not security-audited.

R34 Hermes Projects

HermesLatest/hermes_cli/projects_db.py:23-65,149-178; tui_gateway/contracts/projects_pets.py:64-121; tools/project_tools.py:2-8,75-86; tui_gateway/agent_callbacks.py:194-221. Per-profile workspace organization and session-specific directory behavior, not automatic LiquidAIty Bot Chat isolation.

R35 Governing documents

AGENTS.md, DONT.md, PLAN.md, ARCHITECTURE.md, RUNTIME.md and relevant repository procedures. They were used as requirements/history and checked against source, not assumed to be current implementation proof. Specific contradictory current claims are identified in the report.

R36 Main runtime measurements

HermesLatest/.hermes/profiles/main/logs/agent.log. Baseline 2026-11-01 23:43:23.903 to 23:43:47.193. Rebuild repeat 2026-11-02 00:18:42.782 to 00:18:55.416. Compaction turn 00:21:39.274 to 00:23:35.517. Warm compacted run 00:25:14.727 to 00:25:27.668. Provider counts and compaction telemetry were read directly; not inferred from Engraphis timestamps.

R37 Connected profile measurements

Profile agent.log files under HermesLatest/.hermes/profiles. Builder 00:24:53.513; ThinkGraph 00:26:00.615; KnowGraph 00:27:16.008; card_magentic 00:28:23.197; team 00:29:01.001; trading 00:29:40.397; worldview 00:30:29.964; signal-analyst 00:31:42.291; quant-analyst 00:32:53.058 on 2 November 2026 local time. Each has matching input, API call, completion and dispatch in its own profile log.

R38 Git churn method

Read-only git log --first-parent --since=2026-09-01T00:00:00-04:00 --format=COMMIT markers --numstat --no-renames, grouped by scope and date; October window selected from that history. Binary numstat rows are excluded. Source scope and extensions are defined in Chapter 28. Endpoint git diff uses baseline 6e07220353a2d5db9a5c25aecfba4093ef4e2607. Reflog query since October 1 counted reset entries without inferring reset mode. No Git mutation was performed.

R39 Selected profile configuration

Read-only selected fields from the actual profile config.yaml files. Main threshold_tokens 32000. Main/Builder/Magnetic use gpt-5.6-sol; ThinkGraph/KnowGraph/Trader/WorldView/Analyst/Quant use gpt-5.6-luna; Team uses gpt-5.6-terra in the inspected saved files. No model choice was changed by the audit. Raw absence of an override does not prove every effective inherited runtime value.

R40 Read only database inventory

SQLite opened with URI mode=ro. Hermes Main tables and Engraphis table names were inspected; per-profile message-role counts and file sizes were metadata observations. Project conversation_source count was 67. These were not semantic graph acceptance tests, schema migrations, exports or deletions.

R41 Repository intent skill

.agents/skills/understand-project-design/SKILL.md and agents/openai.yaml; AGENTS.md start-step reference. Owner-authorized instruction-only addition. The bundled skill-creator format validator passed using the existing installed YAML-capable interpreter. A read-only scout evaluated four realistic planning scenarios and reported planning-level PASS, not a guarantee of future implementation compliance.

Appendix B External evidence and research snapshots

Research was performed on 2 November 2026. Repository links are pinned wherever the comparative scout obtained a revision. Maintainer documentation URLs can change after the report date. The evidence supports the particular inspected path, not every deployment or feature of the comparator. No external application was installed or benchmarked for this report.

E01 Official Hermes Desktop

Maintainer README and installed implementation distinguish shared agent state from its UI surfaces. Installed code remains the authority for the app's current integration.

https://github.com/NousResearch/hermes-agent/blob/main/apps/desktop/README.md

https://hermes-agent.nousresearch.com/

E02 Open WebUI sources

Snapshot f7ac7ee10962ddec0017f5c29b6740d9b9c5faca dated 29 October 2026 09:05:24 UTC. E02a model preset schema; E02b user chat schema; E02c memory ownership; E02d context-window behavior; E02e actual compaction; E02f SQL persistence; E02g scaling and task controls; E02h license/performance guidance.

E02a https://github.com/open-webui/open-webui/blob/f7ac7ee10962ddec0017f5c29b6740d9b9c5faca/backend/open_webui/models/models.py#L182

E02b https://github.com/open-webui/open-webui/blob/f7ac7ee10962ddec0017f5c29b6740d9b9c5faca/backend/open_webui/models/chats.py#L180

E02c https://github.com/open-webui/open-webui/blob/f7ac7ee10962ddec0017f5c29b6740d9b9c5faca/backend/open_webui/models/memories.py#L20

E02d https://docs.openwebui.com/troubleshooting/context-window/

E02e https://github.com/open-webui/open-webui/blob/f7ac7ee10962ddec0017f5c29b6740d9b9c5faca/backend/open_webui/utils/context_compaction.py#L45

E02f https://github.com/open-webui/open-webui/blob/f7ac7ee10962ddec0017f5c29b6740d9b9c5faca/backend/open_webui/internal/db.py#L333

E02g https://docs.openwebui.com/getting-started/advanced-topics/scaling/

E02g https://github.com/open-webui/open-webui/blob/f7ac7ee10962ddec0017f5c29b6740d9b9c5faca/backend/open_webui/tasks.py#L32

E02h https://github.com/open-webui/open-webui/blob/f7ac7ee10962ddec0017f5c29b6740d9b9c5faca/LICENSE

E02h https://docs.openwebui.com/troubleshooting/performance/

E03 LibreChat sources

Application snapshot f4154b649efa37e1345ccc517a4d8d1ed2178c82 dated 31 October 2026 19:17:05 UTC. Agents-library snapshot 708c9118f478a84f28e8e253148a373b49fec191 dated 31 October 2026 18:38:45 UTC. E03a and b distinguish agent and conversation; E03c defines retained turns; E03d describes pruning; E03e memory extraction; E03f RAG selection; E03g steering; E03h resumable streams.

E03a https://github.com/danny-avila/LibreChat/blob/f4154b649efa37e1345ccc517a4d8d1ed2178c82/packages/data-schemas/src/schema/agent.ts#L5

E03b https://github.com/danny-avila/LibreChat/blob/f4154b649efa37e1345ccc517a4d8d1ed2178c82/packages/data-schemas/src/schema/convo.ts#L20

E03c https://github.com/LibreChat-AI/agents/blob/708c9118f478a84f28e8e253148a373b49fec191/src/types/summarize.ts#L20

E03d https://github.com/LibreChat-AI/agents/blob/708c9118f478a84f28e8e253148a373b49fec191/docs/summarization-behavior.md

E03d https://github.com/LibreChat-AI/agents/blob/708c9118f478a84f28e8e253148a373b49fec191/src/messages/prune.ts#L849

E03e https://www.librechat.ai/docs/configuration/librechat_yaml/object_structure/memory

E03e https://github.com/danny-avila/LibreChat/blob/f4154b649efa37e1345ccc517a4d8d1ed2178c82/api/server/controllers/agents/client.js#L3600

E03f https://github.com/danny-avila/LibreChat/blob/f4154b649efa37e1345ccc517a4d8d1ed2178c82/packages/api/src/files/rag/search.ts#L75

E03g https://github.com/danny-avila/LibreChat/blob/f4154b649efa37e1345ccc517a4d8d1ed2178c82/api/server/controllers/agents/steer.js#L42

E03h https://www.librechat.ai/docs/features/resumable_streams

E03h https://github.com/danny-avila/LibreChat/blob/f4154b649efa37e1345ccc517a4d8d1ed2178c82/packages/api/src/stream/GenerationJobManager.ts

E04 AnythingLLM sources

Snapshot 9caf2b4c29d9782f1cba4142be2525ebfa0ecd43 dated 30 October 2026 20:49:57 UTC. E04a exchange schema; E04b recent history selection; E04c streaming retrieval; E04d document scope; E04e memory injection/extraction; E04f desktop/server scope; E04g persistence and cancellation ordering.

E04a https://github.com/Mintplex-Labs/anything-llm/blob/9caf2b4c29d9782f1cba4142be2525ebfa0ecd43/server/prisma/schema.prisma#L121

E04b https://github.com/Mintplex-Labs/anything-llm/blob/9caf2b4c29d9782f1cba4142be2525ebfa0ecd43/server/utils/chats/index.js#L83

E04b https://github.com/Mintplex-Labs/anything-llm/blob/9caf2b4c29d9782f1cba4142be2525ebfa0ecd43/frontend/src/pages/WorkspaceSettings/ChatSettings/ChatHistorySettings/index.jsx

E04c https://github.com/Mintplex-Labs/anything-llm/blob/9caf2b4c29d9782f1cba4142be2525ebfa0ecd43/server/utils/chats/stream.js#L149

E04d https://docs.anythingllm.com/chatting-with-documents/introduction

E04e https://github.com/Mintplex-Labs/anything-llm/blob/9caf2b4c29d9782f1cba4142be2525ebfa0ecd43/server/utils/memories/index.js#L17

E04e https://github.com/Mintplex-Labs/anything-llm/blob/9caf2b4c29d9782f1cba4142be2525ebfa0ecd43/server/jobs/extract-memories.js#L17

E04f https://docs.anythingllm.com/installation-desktop/overview

E04g https://github.com/Mintplex-Labs/anything-llm/blob/9caf2b4c29d9782f1cba4142be2525ebfa0ecd43/server/utils/chats/stream.js#L273

E04g https://github.com/Mintplex-Labs/anything-llm/blob/9caf2b4c29d9782f1cba4142be2525ebfa0ecd43/server/utils/helpers/abortSignals.js

E05 Letta sources

Current harness snapshot 3b7e3ffcf16d6ba4880f7a697261837000221320 dated 2 November 2026 00:16:40 UTC. E05a repository relocation notice; E05b independent conversation records; E05c active context versus transcript; E05d MemFS/search; E05e background learning; E05f App Server responsibility.

E05a https://github.com/letta-ai/letta/blob/8816d3a8516893bfdc0fa14bfb1235226012e295/README.md

E05b https://docs.letta.com/concepts/conversations

E05b https://github.com/letta-ai/letta-code/blob/3b7e3ffcf16d6ba4880f7a697261837000221320/src/backend/local/local-conversation-record.ts#L18

E05c https://github.com/letta-ai/letta-code/blob/3b7e3ffcf16d6ba4880f7a697261837000221320/src/backend/local/local-transcript-history.ts

E05c https://github.com/letta-ai/letta-code/blob/3b7e3ffcf16d6ba4880f7a697261837000221320/src/backend/local/compaction.ts#L43

E05d https://docs.letta.com/concepts/memfs

E05d https://github.com/letta-ai/letta-code/blob/3b7e3ffcf16d6ba4880f7a697261837000221320/src/backend/local/transcript-search.ts

E05e https://docs.letta.com/configuration/memory

E05f https://docs.letta.com/self-hosting/app-server

E06 Dify sources

Snapshot ca09f448484d5945fe9852968df903dba2c1cc29 dated 2 November 2026 05:44:29 UTC. E06a history projection; E06b node memory; E06c retrieval; E06d identity scope; E06e deployment; E06f license. Scope is the inspected Chatflow/classic paths, not a complete review of every new Agent subsystem.

E06a https://github.com/langgenius/dify/blob/ca09f448484d5945fe9852968df903dba2c1cc29/api/core/memory/token_buffer_memory.py#L201

E06b https://docs.dify.ai/en/cloud/use-dify/nodes/llm

E06c https://docs.dify.ai/en/cloud/use-dify/nodes/knowledge-retrieval

E06c https://github.com/langgenius/dify/blob/ca09f448484d5945fe9852968df903dba2c1cc29/api/core/rag/datasource/retrieval_service.py#L70

E06d https://docs.dify.ai/en/api-reference/guides/end-user-identity

E06d https://github.com/langgenius/dify/blob/ca09f448484d5945fe9852968df903dba2c1cc29/api/controllers/service_api/app/conversation.py#L193

E06e https://docs.dify.ai/en/self-host/deploy/quick-start/docker-compose

E06f https://github.com/langgenius/dify/blob/ca09f448484d5945fe9852968df903dba2c1cc29/LICENSE

E07 Community Hermes WebUI sources

Snapshot eccc701b9acc73559d76a1baafc6142fd69d9dd6 dated 1 November 2026 23:48:44 UTC. E07a product/execution description; E07b actual runner; E07c session authority; E07d coupling and adapter status; E07e memory-profile scope. This is not official Hermes Desktop.

E07a https://github.com/nesquena/hermes-webui/blob/eccc701b9acc73559d76a1baafc6142fd69d9dd6/README.md

E07b https://github.com/nesquena/hermes-webui/blob/eccc701b9acc73559d76a1baafc6142fd69d9dd6/api/streaming.py#L16431

E07c https://github.com/nesquena/hermes-webui/blob/eccc701b9acc73559d76a1baafc6142fd69d9dd6/api/models.py#L2435

E07c https://github.com/nesquena/hermes-webui/blob/eccc701b9acc73559d76a1baafc6142fd69d9dd6/docs/architecture/unified-session-db.md

E07d https://github.com/nesquena/hermes-webui/blob/eccc701b9acc73559d76a1baafc6142fd69d9dd6/docs/rfcs/agent-source-boundary.md

E07d https://github.com/nesquena/hermes-webui/blob/eccc701b9acc73559d76a1baafc6142fd69d9dd6/api/runtime_adapter.py#L1

E07e https://github.com/nesquena/hermes-webui/blob/eccc701b9acc73559d76a1baafc6142fd69d9dd6/api/memory_profile_scope.py#L1

E10 Retrieval stages and exact identifiers

Primary provider documentation explains dense and sparse retrieval differences, candidate depth and reranking. It supports the conceptual distinction, not a recommendation to install a new store.

https://qdrant.tech/documentation/search-tuning/hybrid-search/

https://qdrant.tech/documentation/tutorials-basics/reranking-hybrid-search/

E11 Contextual retrieval experiments

Anthropic's experiments distinguish initial candidate retrieval and final selected chunks, and report quality differences across chunk counts. Their results are task-specific, not a guaranteed optimum for this application.

https://www.anthropic.com/research/contextual-retrieval

E12 Provider latency guidance

Official guidance identifies output generation, model selection, request count, caching and streaming as distinct latency factors. It does not promise a six-second bound for this account/model or prove an app bottleneck.

https://developers.openai.com/api/docs/guides/latency-optimization

E13 Reasoning controls

Official guidance explains supported reasoning-effort controls and their quality/speed tradeoffs. The report does not infer the effective effort of the measured requests merely from absent raw profile overrides.

https://developers.openai.com/api/docs/guides/reasoning

E14 SQLite deployment boundaries

SQLite's own documentation distinguishes local embedded use, single-file writer coordination and cases where a client/server system is more appropriate. It does not establish a throughput benchmark for LiquidAIty.

https://www.sqlite.org/whentouse.html

https://www.sqlite.org/useovernet.html

E15 Hermes catalog publication

The installed catalog submission and plugin SDK documents were directly read. Public repository references provide the corresponding upstream context; no catalog PR or plugin installation was made.

https://github.com/NousResearch/hermes-agent/tree/main/plugin-catalog

https://github.com/NousResearch/hermes-agent/blob/main/website/docs/developer-guide/plugins/catalog-submission.md

E16 Repository skill discovery

Official documentation describes repository-local .agents/skills discovery and the SKILL.md frontmatter. A single repo-local skill was requested by the owner and added without modifying global Codex configuration.

https://learn.chatgpt.com/docs/build-skills

E18 Retrieval versus long context

The 2024 comparative paper supports a quality/cost tradeoff, not an unconditional claim that RAG always wins. The models and datasets differ from the current application.

https://arxiv.org/abs/2407.16833

E19 Position sensitivity in long input

The Lost in the Middle study motivates evaluating relevance and ordering. It does not prove that every current model fails on every long request.

https://arxiv.org/abs/2307.03172

Appendix C Acceptance matrix for the first release

This matrix is a proposed release test plan, not a report of tests already run. Choose the included feature scope first. Use existing test owners and real saved profiles. Do not create synthetic success records or weaken assertions to match the implementation.

Acceptance case	Observable pass condition	Current evidence
Ordinary Main question	Correct profile receives unchanged input and replies visibly	Observed live on current checkpoint
Each wired direct address	Only the exact connected profile receives the request	Nine connected profiles observed live
Disconnected address	Honest refusal without silently selecting another agent	Source guard present; fresh negative model test not run
Card model or prompt save	Effective Hermes readback and next turn match UI	General parity unverified
Three recent pairs	Actual provider input contains selected complete exchanges not full replay	Not implemented
Old idea recall	Exact authorized old source retrieved with usable surrounding exchange	Search/index source exists; end-to-end acceptance pending
Continuous input	New input accepted through defined Hermes behavior while work exists	Source support; focused live acceptance pending
Stop All	Declared active queued child scope stops and is reflected honestly	Source interrupt path; full scope unproven
Delayed Main delegation	Actual message_agent call recipient work and later foreground reply	Direct tests do not establish this
Completed pair storage	Reply delivered before application save; pair reads back correctly	Eight focused tests and prior readback evidence
Think from new pair	Exact source produces persisted correct Think and citation	Current automatic delivery disconnected
Know research	Verified source plus queued write plus exact persisted readback	Separate live acceptance needed
Graph committed refresh	Correct Project view changes after persistence only	Incomplete source notification wiring
Two concurrent Projects	No conversation roster credential file or retrieval crossing	Not proven; source coupling identified
Two authenticated users	Separate private profile and data authority	Not provisioned/proven by current global profile namespace
Project deletion/export	Exact documented effects across all data owners	Current deletion narrower than full-data promise
Fresh supported machine	Full product starts and included workflow works without hidden steps	Current workstation only in this pass

Acceptance case	Observable pass condition	Current evidence
Recovery	Authorized restore recreates usable intended state	Not performed by audit
Public abuse/privacy controls	Tested identity recovery limits private services and scope failures	Public gate remains open
Performance	Agreed first-text/completion p50/p95 under defined workload	Single-run measurements only; Main target not met

Appendix D Audit change record and limitations

The reporting stage authored the official report source, PDF/text generation utility, final report outputs and full catalog inventory. The owner separately requested an understanding skill. The authored repository instructions are .agents/skills/understand-project-design/SKILL.md, its optional interface metadata, and the small AGENTS routing reference. No production TypeScript/Python runtime, Hermes source, provider settings or graph implementation was changed for this report.

The current 32000 Main compaction trigger was set in the preceding authorized configuration step. Three-pair selection remains a proposed implementation direction, not a setting already active. The report deliberately does not undo that configuration under a new assumption. Any change to it should be explicit in the next approved stage.

The existing full application process was left running and Preview retained. The nine connected-agent checks and repeated Main questions occurred earlier in the same review conversation under explicit owner authorization. Their resulting conversation writes are expected product effects, not report-driven database edits. No additional model test was sent to make the report look more complete.

The source scouts worked read-only. Their results were reconciled into this report rather than copied into separate competing architecture files. Research snapshots are documented separately from the installed source. No dependency, plugin, account, permission or deployment was added. No Git reset, restore, clean, stage, commit, push, branch or fetch was performed in this stage.

The audit does not establish production security, every possible tool workflow, every historical feature idea, semantic graph quality for all conversations, a formal concurrency proof, or performance at the user's aspirational scale. Those limitations are explicit because the purpose of this report is to enable an independent reviewer to identify the next real work, not to conceal it under a blanket passing status.

The central conclusion remains: preserve the now-demonstrated chat foundation; complete specific missing product boundaries; make decisions at the right stage; and require actual evidence before declaring each stage finished.